

Лабораторная работа № 5. Построение графиков

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Предварительные сведения	8
3.1.1	Основные пакеты для работы с графиками в Julia	8
3.1.2	Опции при построении графика	13
3.1.3	Точечный график	16
3.1.4	Аппроксимация данных	18
3.1.5	Две оси ординат	20
3.1.6	Полярные координаты	22
3.1.7	Параметрический график	22
3.1.8	График поверхности	24
3.1.9	Линии уровня	26
3.1.10	Векторные поля	28
3.1.11	Анимация	31
3.1.12	Errorbars	37
3.1.13	Использование пакета Distributions	41
3.1.14	Подграфики	44
3.2	Задания для самостоятельного выполнения	48
4	Выводы	59

Список иллюстраций

3.1	Задание исходной функции	9
3.2	График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи <code>gr()</code>	10
3.3	График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи <code>subplot()</code>	11
3.4	График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи <code>plotly()</code>	12
3.5	График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи <code>unicodeplots()</code>	12
3.6	График функции $\sin(x)$	13
3.7	График функции разложения исходной функции в ряд Тейлора	14
3.8	Графики исходной функции и её разложения в ряд Тейлора	14
3.9	Вид графиков после добавления опций при их построении	15
3.10	Сохранение построенного графика	15
3.11	График десяти случайных значений на плоскости	16
3.12	График пятидесяти случайных значений на плоскости с различными опциями отображения	17
3.13	График пятидесяти случайных значений в пространстве с различными опциями отображения	18
3.14	Пример функции	19
3.15	Пример аппроксимации исходной функции полиномом 5-й степени	20
3.16	Пример отдельно построенной траектории	21
3.17	Пример двух траекторий на одном графике с двумя осями ординат	21
3.18	График функции, заданной в полярных координатах	22
3.19	Параметрический график кривой на плоскости	23
3.20	Параметрический график кривой в пространстве	24
3.21	График поверхности (использована функция <code>surface()</code>)	24
3.22	График поверхности (использована функция <code>plot()</code>)	25
3.23	Сглаженный график поверхности	25
3.24	График поверхности с изменённым углом зрения	26
3.25	График поверхности, заданной функцией $g(x, y) = (3x + y^2) \sin(x) + \cos(y) $	27
3.26	Линии уровня	27
3.27	Линии уровня с заполнением	28
3.28	График функции $h(x, y) = x^3 - 3x + y^2$	29
3.29	Линии уровня для функции $h(x, y) = x^3 - 3x + y^2$	29
3.30	Векторное поле функции $h(x, y) = x^3 - 3x + y^2$	30

3.31 Векторное поле функции $h(x, y) = x^3 - 3x + y^2$ (скорректирована область видимости)	30
3.32 Статичный график поверхности	31
3.33 Анимированный график поверхности	32
3.34 Задание параметров гипоциклоиды	33
3.35 Большая окружность гипоциклоиды с осями координат	34
3.36 Половина пути гипоциклоиды и малая окружность	35
3.37 Малая окружность гипоциклоиды с добавлением радиуса	36
3.38 Анимация движения гипоциклоиды	36
3.39 Анимация движения гипоциклоиды	37
3.40 Задание массива значений и генерация массива ошибок	38
3.41 Графики исходных значений и исходных значений с отклонениями	39
3.42 Поворот графика	40
3.43 Заполнение цветом	40
3.44 График ошибок по двум осям	41
3.45 График асимметричных ошибок по двум осям	41
3.46 Гистограмма, построенная по массиву случайных чисел	42
3.47 Гистограмма нормального распределения	43
3.48 Гистограмма распределения людей по возрастам	43
3.49 Серия из 4-х графиков в ряд и в сетке	44
3.50 Серия из 4-х графиков разной высоты в ряд	45
3.51 Объединение нескольких графиков в одной сетке	46
3.52 Разнообразные варианты представления данных	47
3.53 Демонстрация применения сложного макета для построения графиков	47
3.54 Все возможные типы графиков функции $y = \sin(x)$, где $x \in [0, 2\pi]$.	48
3.55 Демонстрация всех типов линий графика для функции $y = \sin(x)$, где $x \in [0, 2\pi]$	49
3.56 График функции $y(x) = \pi x^2 \ln(x)$	50
3.57 Графическое изображение по точкам x значения функции $y(x) = x^3 - 3x$ в разном виде	51
3.58 Первое построение графика	52
3.59 Второе построение графика	52
3.60 График экспериментальных данных с учетом ошибки измерения .	53
3.61 Точечный график случайных данных	54
3.62 3-мерный точечный график случайных данных	55
3.63 Анимация построения синусоиды	56
3.64 Анимация гипоциклоиды	57
3.65 Анимация эпициклоида	58

Список таблиц

1 Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

2 Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

3 Выполнение лабораторной работы

3.1 Предварительные сведения

3.1.1 Основные пакеты для работы с графиками в Julia

Julia поддерживает несколько пакетов для работы с графиками. Использование того или иного пакета зависит от целей, преследуемых пользователем при построении. Стандартным для Julia является пакет `Plots.jl`. Перед использованием графических пакетов следует их установить и подключить в Julia:

```
using Pkg
Pkg.add("Plots")
Pkg.add("PyPlot")
Pkg.add("Plotly")
Pkg.add("UnicodePlots")
# подключаем для использования Plots:
using Plots
```

Одним из преимуществ `Plots.jl` является то, что он позволяет легко менять вызов библиотек работы с графикой: `gr()`, `pyplot()`, `plotlyjs()`. Далее рассмотрим построение графика функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ разными способами. Фактически для построения графика функции требуется иметь массив соответствующих значений x и y . Зададим исходную функцию (используются поэлементные операции над векторами)(рис. 3.1):

```

In [2]: # задание функции:
f(x) = (3x.^2 + 6x - 9).*exp.(-0.3x)

Out[2]: f (generic function with 1 method)

In [3]: # генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1
# (шаг задан через указание длины массива):
x = collect(range(-5,10,length=151))

Out[3]: 151-element Vector{Float64}:
 -5.0
 -4.9
 -4.8
 -4.7
 -4.6
 -4.5
 -4.4
 -4.3
 -4.2
 -4.1
 -4.0
 -3.9
 -3.8
  ⋮
  8.9
  9.0
  9.1
  9.2
  9.3
  9.4
  9.5
  9.6
  9.7
  9.8
  9.9
 10.0

In [4]: # генерирование массива значений y:
y = f(x)

Out[4]: 151-element Vector{Float64}:
161.34080653217032
146.26477779394003
132.19219298833204
118.05042250634011

```

Рис. 3.1: Задание исходной функции

Далее показано сравнение построения графиков при использовании разных программно-аппаратных частей Plots.jl. По умолчанию используется `gr()` (рис. 3.2):

```
In [5]: # указывается, что для построения графика используется gr():  
gr()
```

```
Out[5]: Plots.GRBackend()
```

```
In [6]: # задание опций при построении графика  
# (название кривой, подписи по осям, цвет графика):  
plot(x,y,  
      title="A simple curve",  
      xlabel="Variable x",  
      ylabel="Variable y",  
      color="blue")
```

```
Out[6]:
```

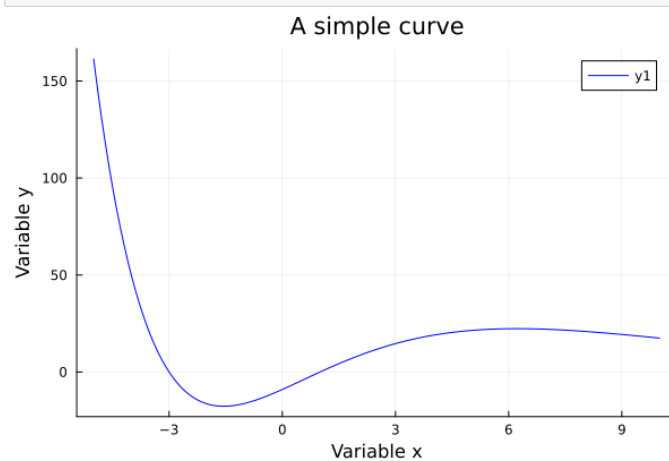


Рис. 3.2: График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи `gr()`

Если требуется, чтобы на графике были надписи на русском языке, то лучше воспользоваться `ruplot()` (рис. 3.3):

```
In [7]: # указывается, что для построения графика используется pyplot():
pyplot()
# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,|
      title="Простая кривая",
      xlabel="Переменная x",
      ylabel="Переменная y",
      color="blue")
```

Out[7]:

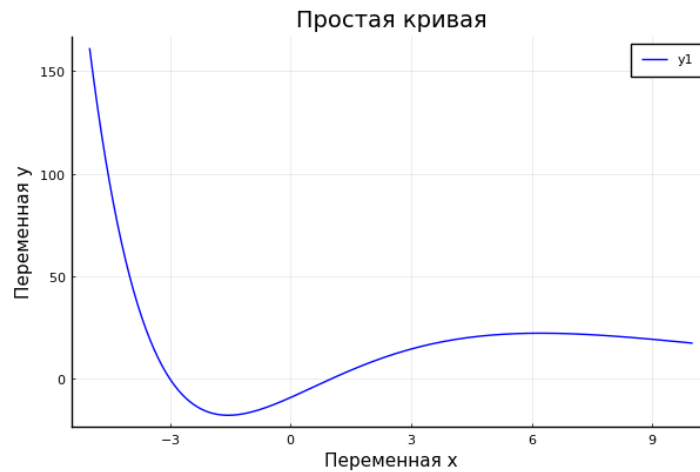


Рис. 3.3: График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи `pyplot()`

В качестве упражнения построим график функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ при помощи `plotly()` и `unicodeplots()` (рис. 3.4):

```
In [17]: # указывается, что для построения графика используется pyplot():
plotly()
# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,
      title="Простая кривая",
      xlabel="Переменная x",
      ylabel="Переменная y",
      color="blue")
```

Out[17]:

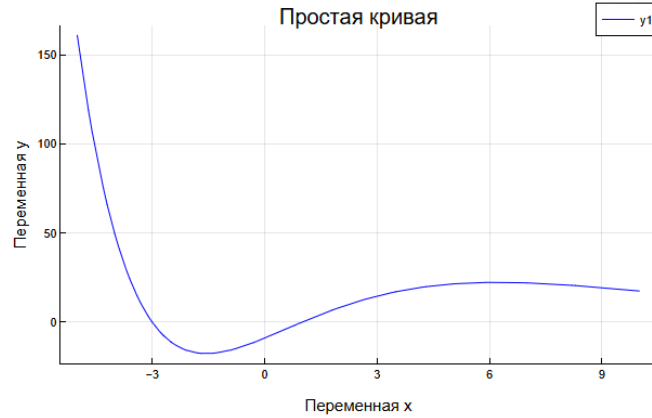


Рис. 3.4: График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи plotly()

```
In [15]: # указывается, что для построения графика используется pyplot():
unicodeplots()
# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,
      title="Простая кривая",
      xlabel="Переменная x",
      ylabel="Переменная y",
      color="blue")
```

Out[15]:



Рис. 3.5: График функции $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$, построенный при помощи unicodeplots()

3.1.2 Опции при построении графика

Далее на примере графика функции $\sin(x)$ и графика разложения этой функции в ряд Тейлора:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}, \quad x \in \mathbb{C}$$

рассмотрим дополнительные возможности пакетов для работы с графикой. Используем `pyplot()` (рис. 3.6):

```
In [19]: # указывается, что для построения графика используется pyplot():
          pyplot()
          # Задаём функцию:
          # задание функции sin(x):
          sin_theor(x) = sin(x)
          # Строим график функции (рис. 5.3):
          # построение графика функции sin(x):
          plot(sin_theor)
```

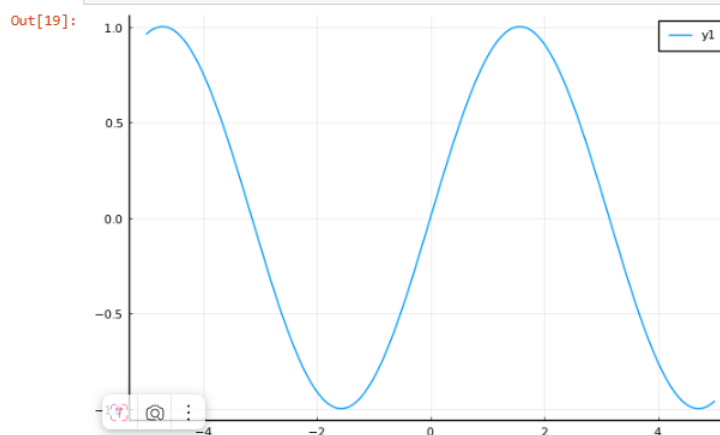


Рис. 3.6: График функции $\sin(x)$

Задаём разложение исходной функции в ряд Тейлора. Строим график разложения исходной функции в ряд Тейлора (рис. 3.7):

```
In [20]: # задание функции разложения исходной функции в ряд Тейлора:
sin_taylor(x) = [(-1)**i*x**(2*i+1)/factorial(2*i+1) for i in 0:4] |> sum
# Строим график разложения исходной функции в ряд Тейлора (рис. 5.4):
# построение графика функции sin_taylor(x):
plot(sin_taylor)
```

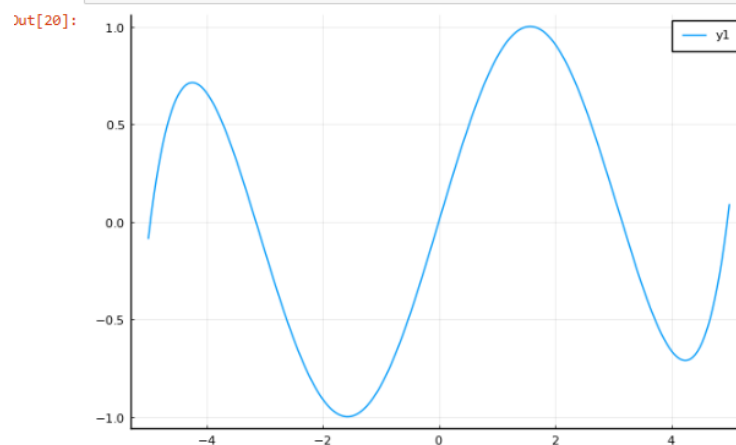


Рис. 3.7: График функции разложения исходной функции в ряд Тейлора

Выводим две функции на один график (рис. 3.8):

```
In [21]: # построение двух функций на одном графике:
plot(sin_theor)
plot!(sin_taylor)
```

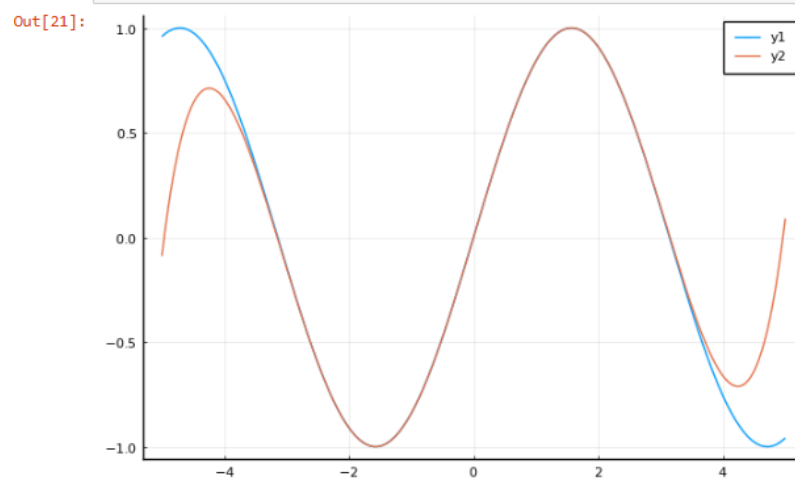


Рис. 3.8: Графики исходной функции и её разложения в ряд Тейлора

Далее добавим различные опции для отображения на графике (рис. 3.9):

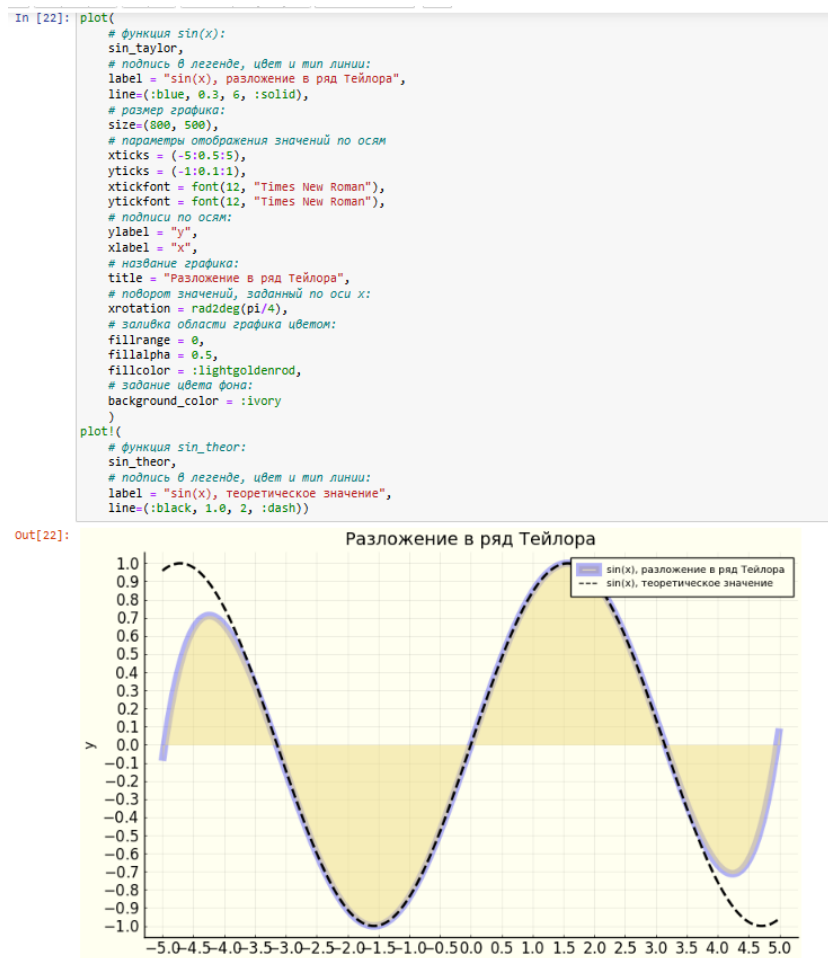


Рис. 3.9: Вид графиков после добавления опций при их построении

Для сохранения построенного графика в определённом формате можно использовать функцию `savefig()` (рис. 3.10):

```

In [23]: # сохранение графика в файле в формате pdf или png:
savefig("taylor.pdf")
savefig("taylor.png")

```

Out[23]: "C:\\Users\\user\\Downloads\\taylor.png"

Рис. 3.10: Сохранение построенного графика

3.1.3 Точечный график

Графики в виде точек на плоскости или в пространстве часто используются в статистических исследованиях.

3.1.3.1 Простой точечный график

Как и построении обычного графика для точечного графика необходимо задать массив значений x , посчитать или задать значения y , задать опции построения графика (рис. 3.11):

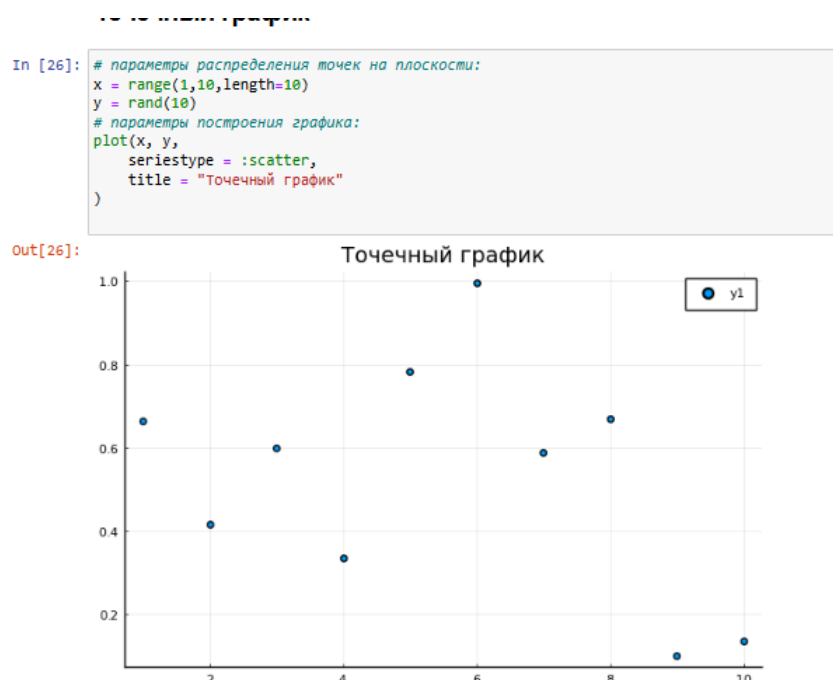


Рис. 3.11: График десяти случайных значений на плоскости

3.1.3.2 Точечный график с кодированием значения размером точки

Для точечного графика можно задать различные опции, например размер маркера, его тип, цвет и и т.п (рис. 3.12):

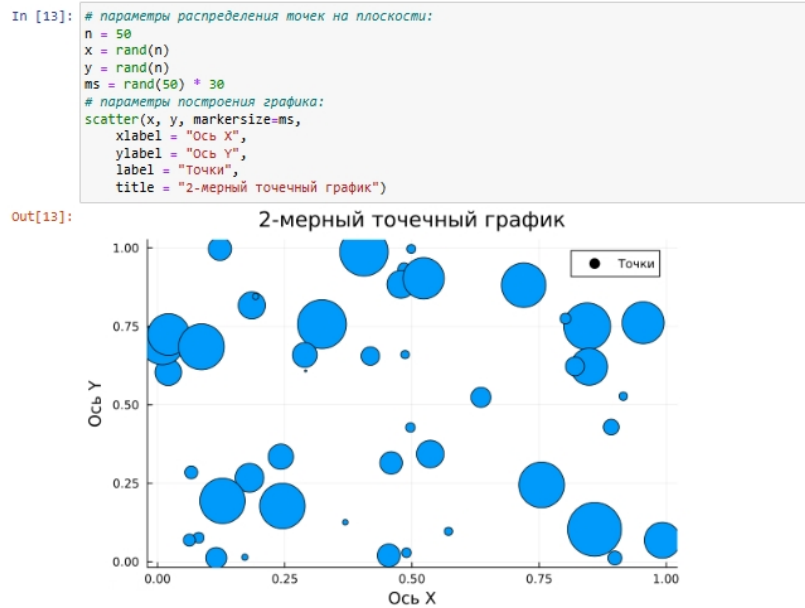


Рис. 3.12: График пятидесяти случайных значений на плоскости с различными опциями отображения

3.1.3.3 3-мерный точечный график с кодированием значения размером точки

Можно строить и 3-мерные точечные графики (рис. 3.13):

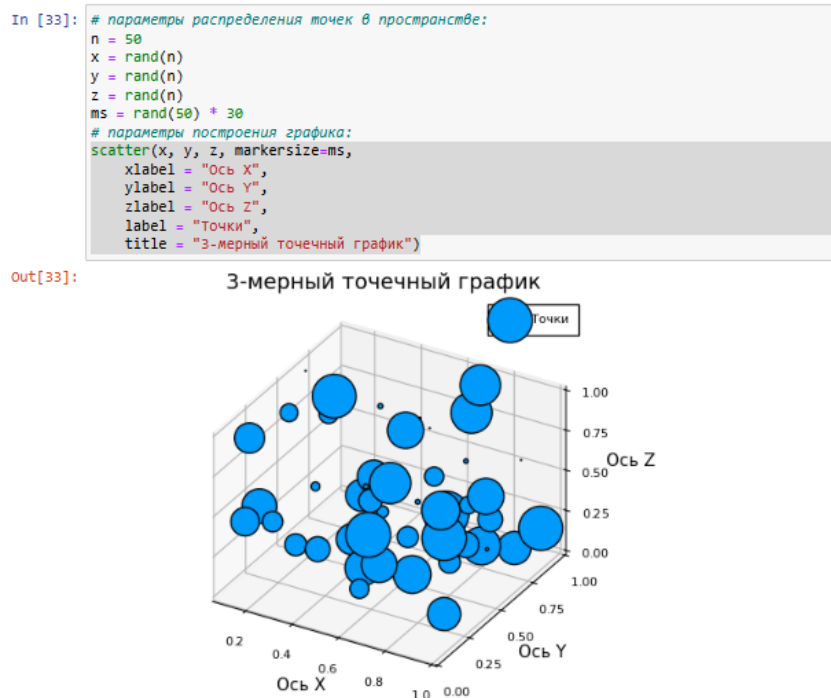


Рис. 3.13: График пятидесяти случайных значений в пространстве с различными опциями отображения

Также выше в качестве упражнения был дополнен код построения точечных графиков, подписаны оси, добавлены легенду и название (рис. 3.12) и (рис. 3.13).

3.1.4 Аппроксимация данных

Аппроксимация — научный метод, состоящий в замене объектов их более простыми аналогами, сходными по своим свойствам. Пусть на некотором отрезке в точках $(x_0, x_1, x_2, \dots, x_N)$ известны значения некоторой функции $f(x)$, а именно $(y_0, y_1, y_2, \dots, y_N)$. Требуется определить параметры a_i многочлена вида вида $F(x) = a_0 + a_1x + \dots + a_kx^k$, де $k < N$ такое, что сумма квадратов отклонений значений y от значений функции $F(y)$ в заданных точках x минимальна, т.е.

$$S = \sum_{i=0}^N [y_i - F(x_i, a_0, a_1, \dots, a_k)]^2 \rightarrow \min$$

Геометрически это означает, что нужно найти кривую $y = F(x)$, полином кото-

рой проходит как можно ближе к каждой из заданных точек. Такая задача может быть решена, если решить систему уравнений вида:

$$A\vec{x} = \vec{y},$$

где A — матрица коэффициентов многочлена $F(x)$, а $\vec{x}$ и $\vec{y}$ — векторы соответствующих значений. Для демонстрации зададим искусственно некоторую функцию, в данном случае похожую по поведению на экспоненту (рис. 3.14):

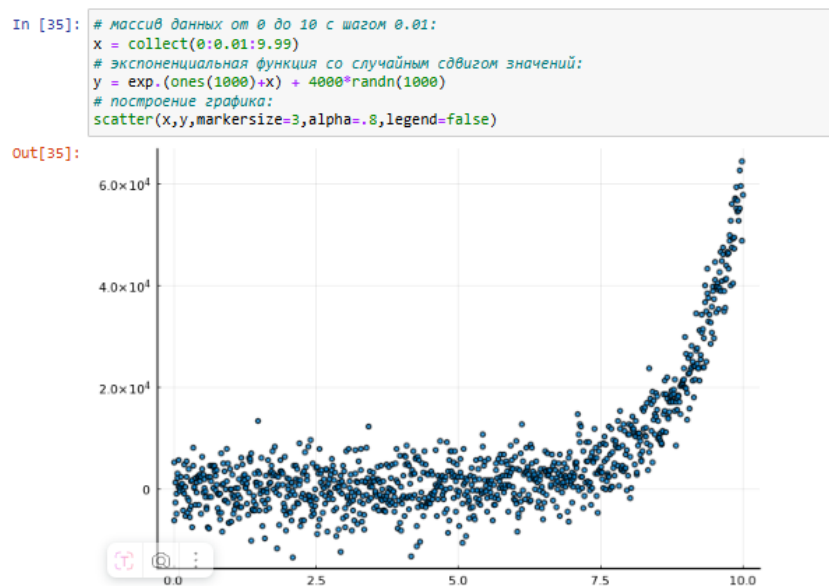


Рис. 3.14: Пример функции

Аппроксимируем полученную функцию полиномом 5-й степени (рис. 3.15):

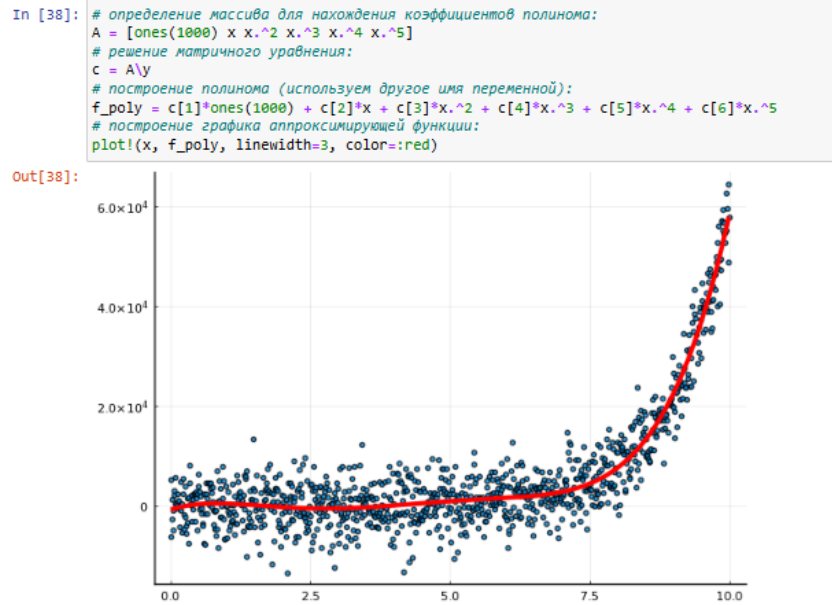


Рис. 3.15: Пример аппроксимации исходной функции полиномом 5-й степени

3.1.5 Две оси ординат

Иногда требуется на один график вывести несколько траекторий с существенными отличиями в значениях по оси ординат. Пример первой траектории(рис. 3.16):

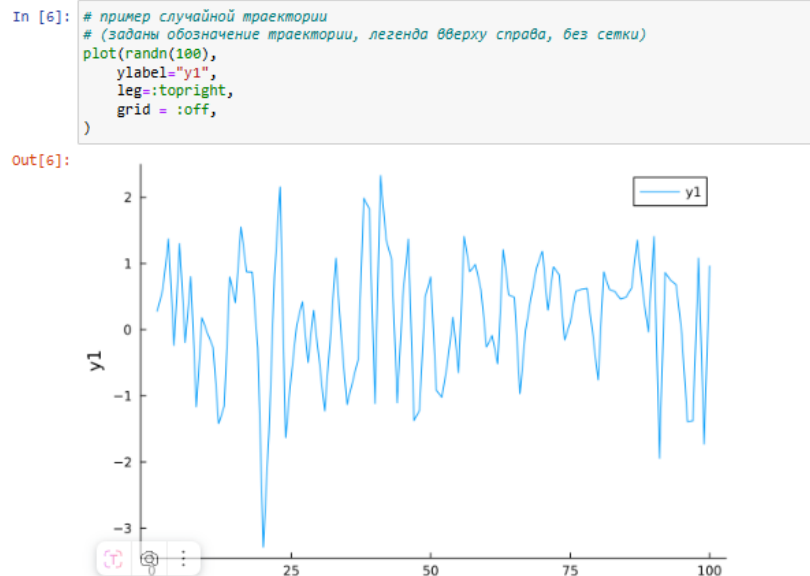


Рис. 3.16: Пример отдельно построенной траектории

Далее на существующий график добавляется вторая траектория с дополнительными элементами для улучшения восприятия информации(рис. 3.17):

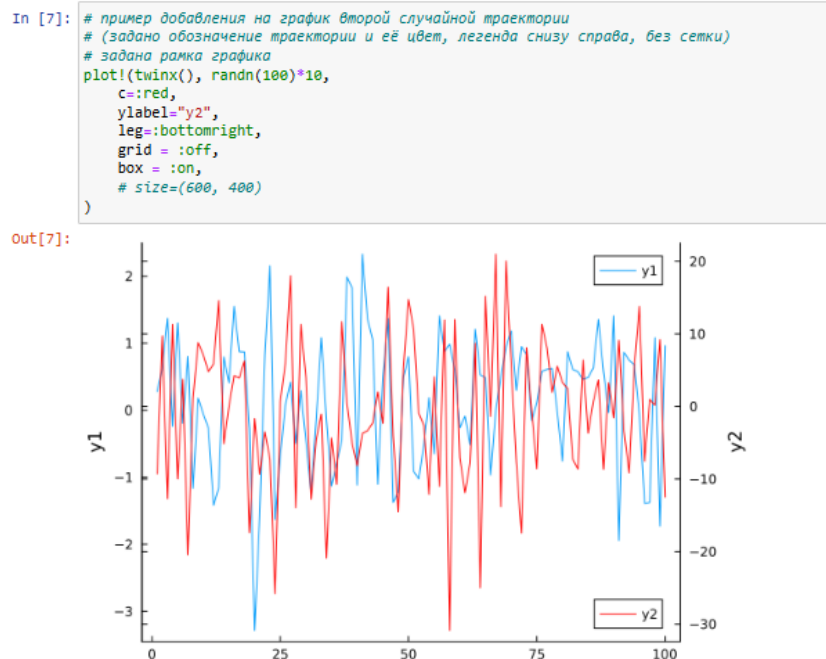


Рис. 3.17: Пример двух траекторий на одном графике с двумя осями ординат

3.1.6 Полярные координаты

Приведём пример построения графика функции

$$r(\vartheta) = 1 + \cos \vartheta \cdot \sin^2 \vartheta$$

в полярных координатах (рис. 3.18):

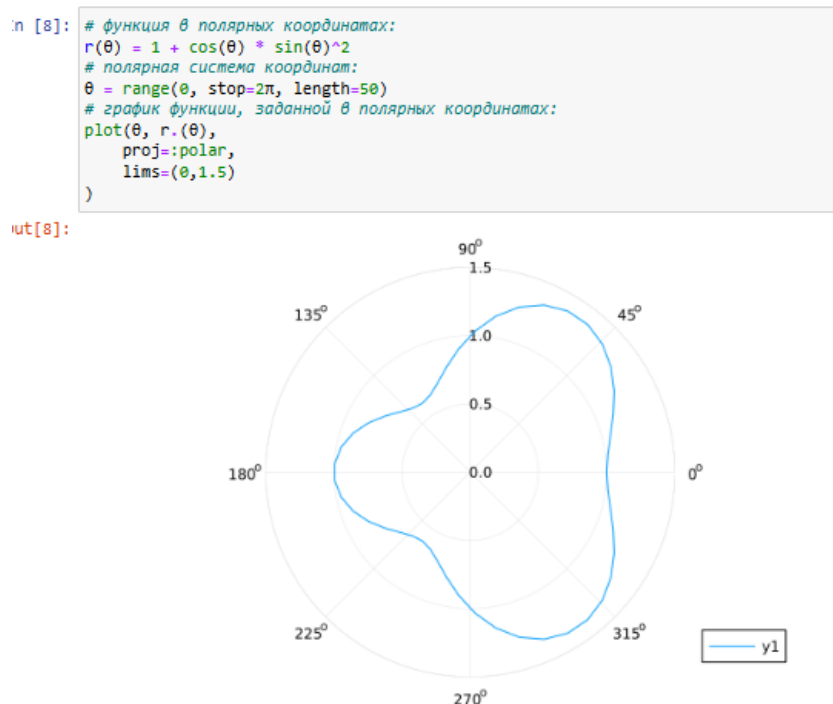


Рис. 3.18: График функции, заданной в полярных координатах

3.1.7 Параметрический график

При параметрическом представлении графика некоторой функции координаты на графике задаются как функции от некоторого набора свободных параметров. В случае одного параметра получим параметрическое уравнение кривой. Выражая координаты точек поверхности через два свободных параметра, получим параметрическое задание поверхности.

3.1.7.1 Параметрический график кривой на плоскости

Приведём пример построения графика (рис. 3.19) параметрически заданной кривой на плоскости. Кривая задана выражениями

$$\begin{aligned}x(t) &= \sin(t), \\ y(t) &= \sin(2t), \quad 0 \leq t \leq 2\pi\end{aligned}$$

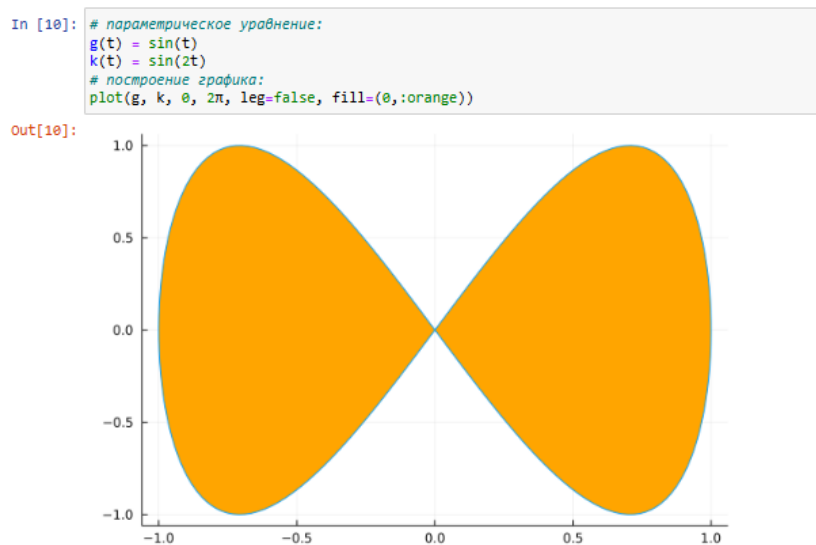


Рис. 3.19: Параметрический график кривой на плоскости

3.1.7.2 Параметрический график кривой в пространстве

Приведём пример построения графика (рис. 3.20) параметрически заданной кривой в пространстве. Кривая задана выражениями:

$$x(t) = \cos(t), \quad y(t) = \sin(t), \quad z(t) = \sin(5t).$$

```
In [12]: t = range(0, stop=10, length=1000)
g_val = cos.(t)
k_val = sin.(t)
p_val = sin.(5t)

plot(g_val, k_val, p_val)
```

Out[12]:

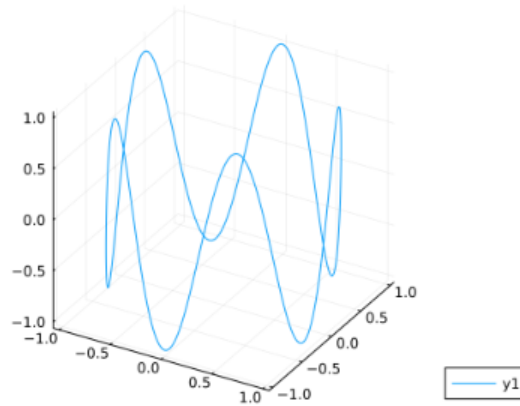


Рис. 3.20: Параметрический график кривой в пространстве

3.1.8 График поверхности

Для построения поверхности, заданной уравнением $f(x, y) = x^2 + y^2$, можно воспользоваться функцией `surface()` (рис. 3.21):

```
In [2]: # построение графика поверхности:
f(x,y) = x^2 + y^2
x = -10:10
y = x
surface(x, y, f)
```

Out[2]:

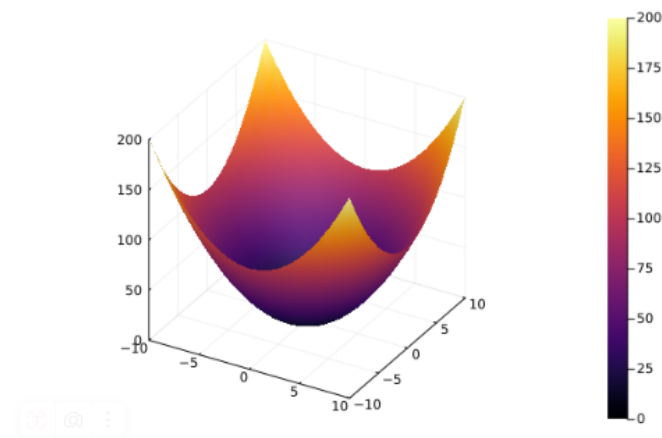


Рис. 3.21: График поверхности (использована функция `surface()`)

Также можно воспользоваться функцией `plot()` с заданными параметрами (рис. 3.22):

```
In [3]: # построение графика поверхности:  
f(x,y) = x^2 + y^2  
x = -10:10  
y = x  
plot(x, y, f,  
      linestyle=:wireframe  
)
```

out[3]:

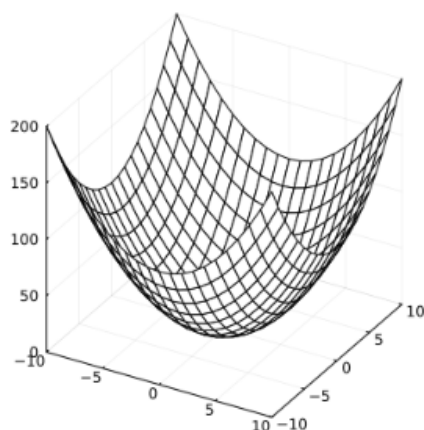


Рис. 3.22: График поверхности (использована функция `plot()`)

Можно задать параметры сглаживания (рис.3.23):

```
In [4]: f(x,y) = x^2 + y^2  
x = -10:0.1:10  
y = x  
plot(x, y, f,  
      linestyle = :surface  
)
```

out[4]:

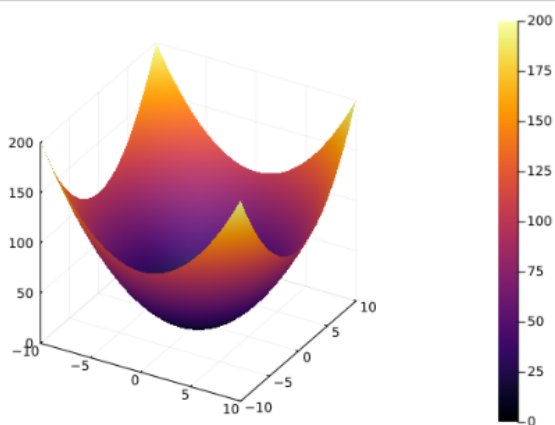


Рис. 3.23: Сглаженный график поверхности

Можно задать определённый угол зрения (рис.3.24):

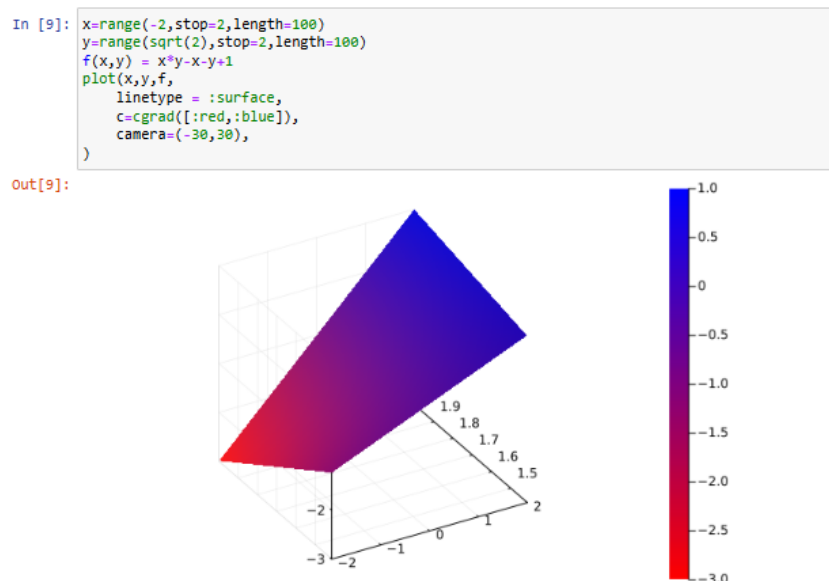


Рис. 3.24: График поверхности с изменённым углом зрения

3.1.9 Линии уровня

Линией уровня некоторой функции от двух переменных называется множество точек на координатной плоскости, в которых функция принимает одинаковые значения. Линий уровня бесконечно много, и через каждую точку области определения можно провести линию уровня.

С помощью линий уровня можно определить наибольшее и наименьшее значение исходной функции от двух переменных. Каждая из этих линий соответствует определённому значению высоты.

Поверхности уровня представляют собой непересекающиеся пространственные поверхности.

Рассмотрим поверхность (рис.3.25), заданную функцией

$$g(x,y) = (3x + y^2) |\sin(x) + \cos(y)|$$

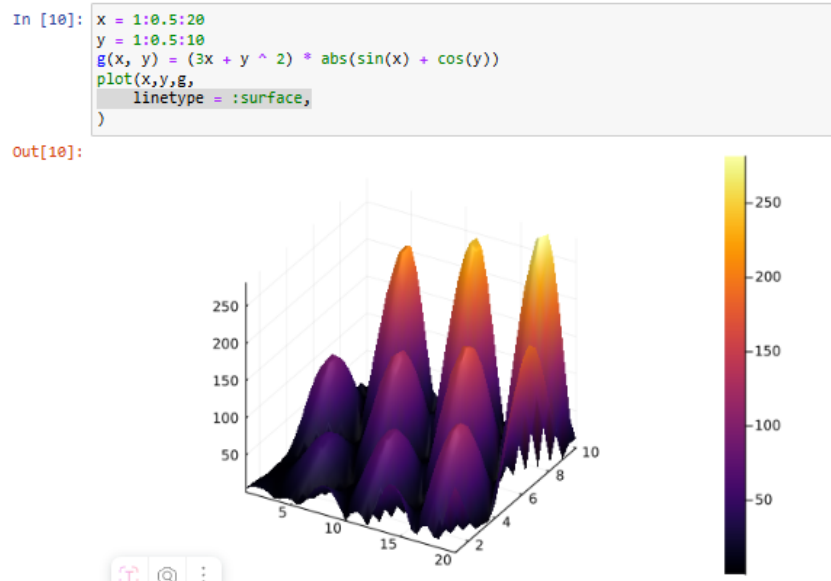


Рис. 3.25: График поверхности, заданной функцией $g(x, y) = (3x + y^2) |\sin(x) + \cos(y)|$

Линии уровня можно построить, используя проекцию значений исходной функции на плоскость (рис.3.26):

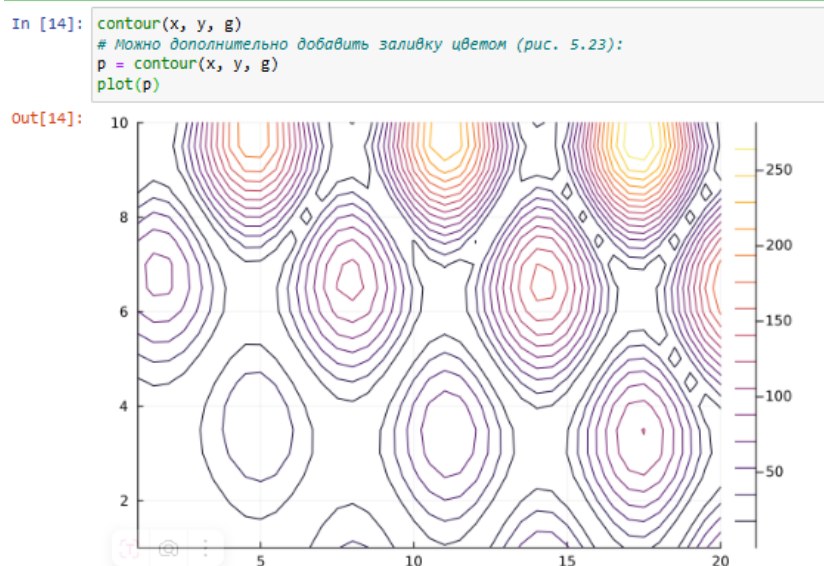


Рис. 3.26: Линии уровня

Можно дополнительно добавить заливку цветом (рис.3.27):

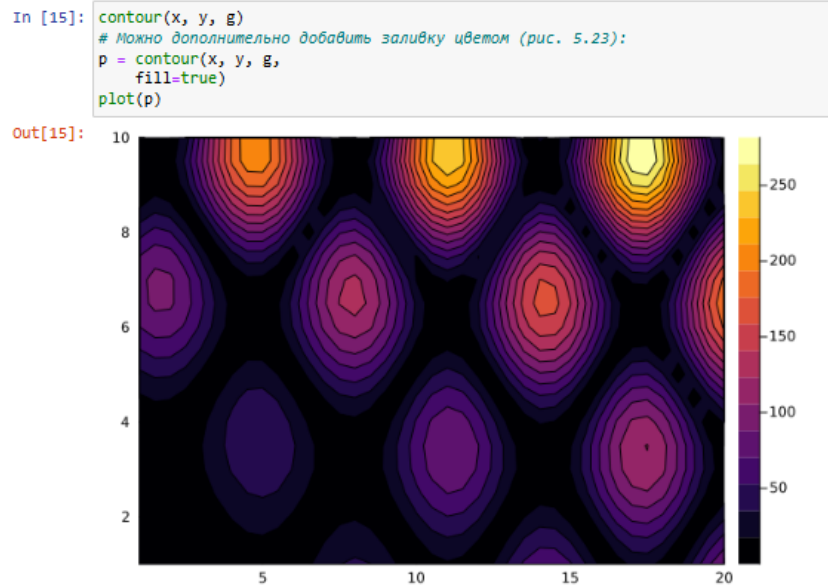


Рис. 3.27: Линии уровня с заполнением

3.1.10 Векторные поля

Если каждой точке некоторой области пространства поставлен в соответствие вектор с началом в данной точке, то говорят, что в этой области задано векторное поле.

Векторные поля задают векторными функциями. Для функции $h(x, y) = x^3 - 3x + y^2$ сначала построим её график (рис.3.28) и линии уровня (рис.3.29):

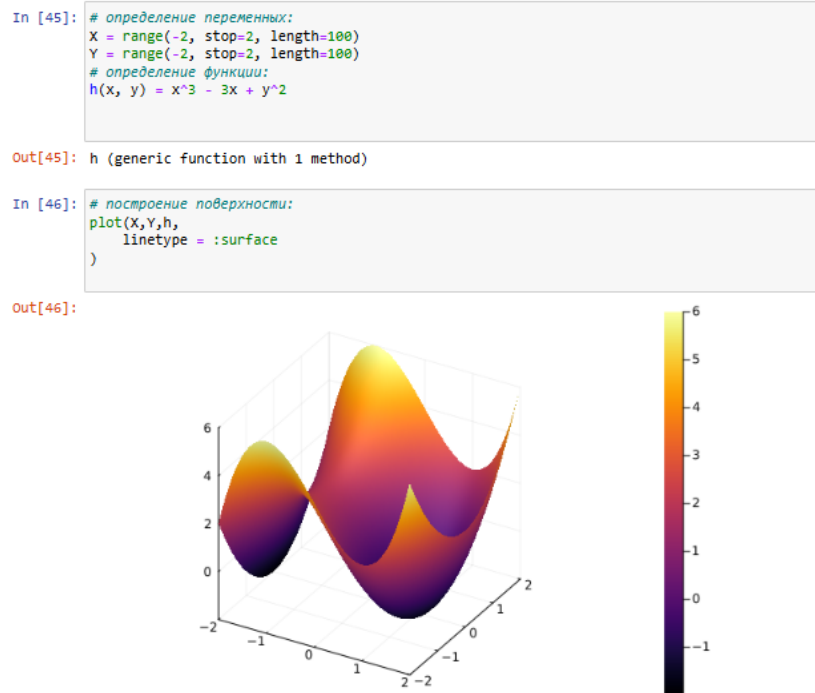


Рис. 3.28: График функции $h(x, y) = x^3 - 3x + y^2$

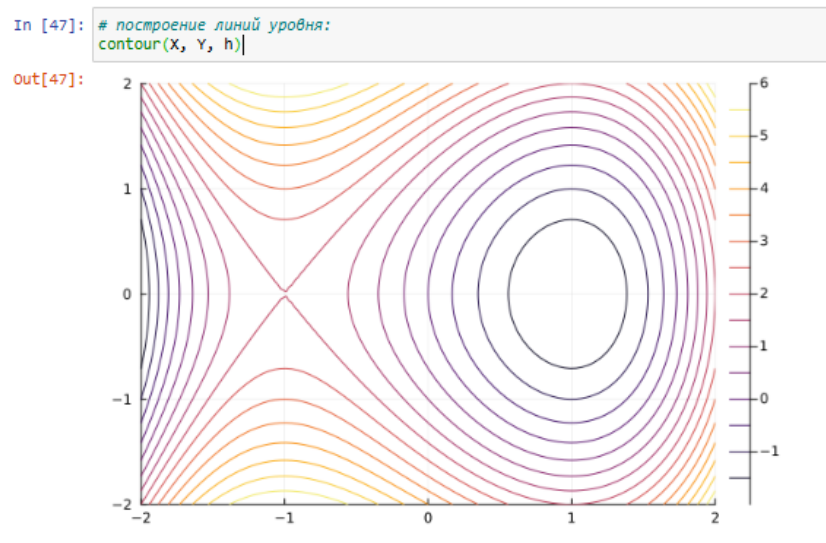


Рис. 3.29: Линии уровня для функции $h(x, y) = x^3 - 3x + y^2$

Векторное поле можно охарактеризовать векторными линиями. Каждая точка векторной линии является началом вектора поля, который лежит на касательной в данной точке.

Для заданной функции построим векторное поле (рис.3.30):

```
In [50]: h(x, y) = x^3 - 3x + y^2
dh(x, y) = ((3x^2 - 3)/25, (2y)/25) # КОРТЕЖ, не вектор!

p = contour(x, y, h, levels=20, fill=false)

# Сетка для стрелок (меньше точек)
xs = range(-2, stop=2, length=12)
ys = range(-2, stop=2, length=12)
Xq = vec([x for x in xs, y in ys])
Yq = vec([y for x in xs, y in ys])

quiver!(Xq, Yq, quiver=dh, color=:blue)
display(p)
```

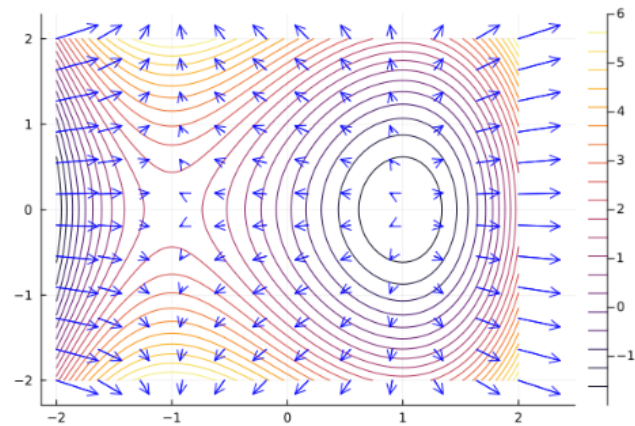
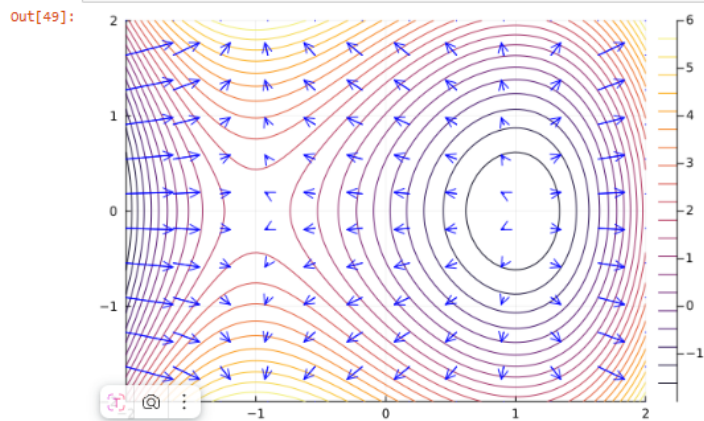


Рис. 3.30: Векторное поле функции $h(x, y) = x^3 - 3x + y^2$

```
In [49]: xlims!(-2, 2)
          ylims!(-2, 2)
```



Анимация

Рис. 3.31: Векторное поле функции $h(x, y) = x^3 - 3x + y^2$ (скорректирована область видимости)

Для получения рис.3.30 и рис.3.31 был изменен листинг.

3.1.11 Анимация

Технически анимированное изображение представляет собой несколько наложенных изображений (или построенных в разных точках графиках) в одном файле.

3.1.11.1 Gif-анимация

В Julia рекомендуется использовать gif-анимацию в `pyplot()` (рис.3.32):

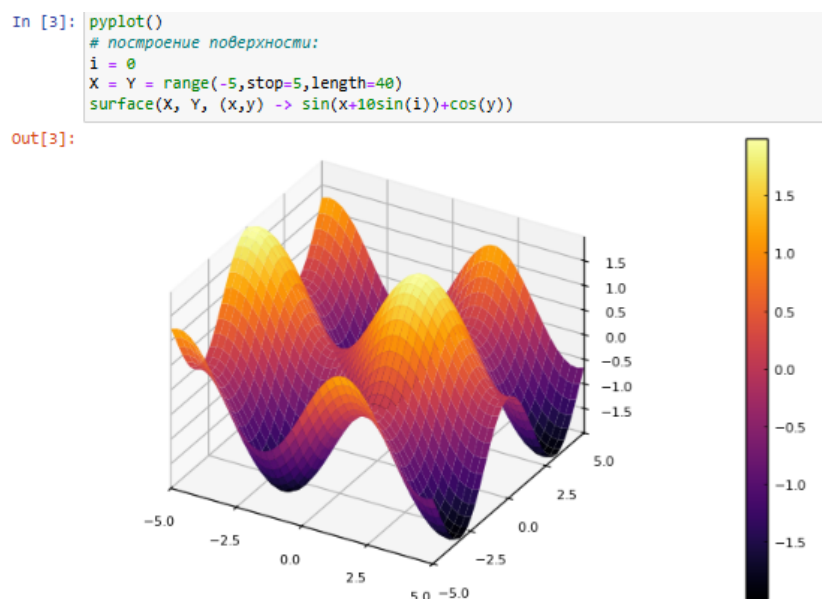


Рис. 3.32: Статичный график поверхности

Добавляем анимацию (рис.3.33):

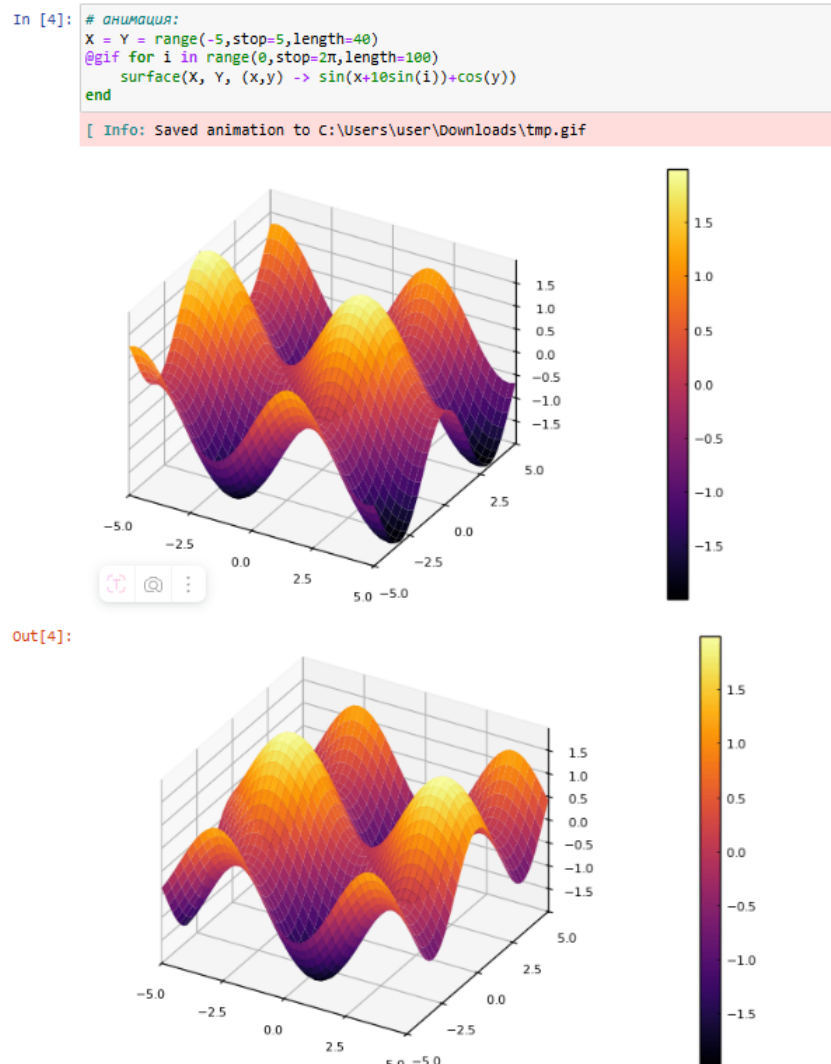


Рис. 3.33: Анимированный график поверхности

3.1.11.2 Гипоциклоида

Гипоциклоида — плоская кривая, образуемая точкой окружности, катящейся по внутренней стороне другой окружности без скольжения:

$$\begin{aligned}
 x &= r(k-1) \left(\cos t + \frac{\cos((k-1)t)}{k-1} \right), \\
 y &= r(k-1) \left(\sin t - \frac{\sin((k-1)t)}{k-1} \right),
 \end{aligned}$$

где $k = \frac{R}{r}$, R — радиус неподвижной окружности, r — радиус катящейся окружности. Модуль величины k определяет форму гипоциклоиды. Построим гипоциклоиду. Сначала зададим параметры (рис.3.34):

```
In [5]: # радиус малой окружности:
r = 1
# коэффициент для построения большой окружности:
k = 3
# число отсчётов:
n = 100
```

Out[5]: 100

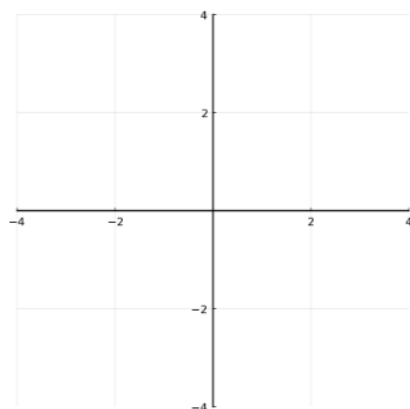
```
In [7]: # массив значений угла θ:
# theta from 0 to 2π ( + a little extra)
θ = collect(0:2*π/100:2*π+2*π/100)
# массивы значений координат:
X = r*k*cos.(θ)
Y = r*k*sin.(θ)
```

```
Out[7]: 101-element Vector{Float64}:
 0.0
 0.18837155858794014
 0.3759997006929128
 0.5621439437571739
 0.7460696614945643
 0.9270509831248421
 1.104373658054034
 1.2773378746952182
 1.445261022305146
 1.60748038493699
 1.7633557568774194
 1.9122719692460692
 2.053641317786066
 ⋮
-1.9122719692460688
-1.7633557568774179
-1.607480384936989
-1.445261022305146
-1.2773378746952166
-1.1043736580540335
-0.9270509831248404
-0.7460696614945634
-0.562143943757174
-0.37599970069291133
-0.1883715585879398
 1.929747179611964e-15
```

Рис. 3.34: Задание параметров гипоциклоиды

Построим оси координат и большую окружность (рис.3.35):

```
In [8]: # задаём оси координат:
plt=plt.plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1, legend=false, framestyle=:origin)
Out[8]:
```



```
In [9]: # большая окружность:
plot!(plt, X,Y, c=:blue, legend=false)
Out[9]:
```

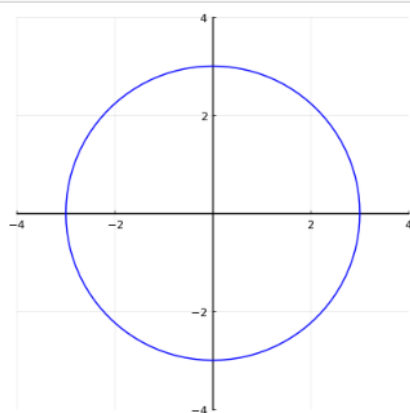


Рис. 3.35: Большая окружность гипоциклоиды с осями координат

Для частичного построения гипоциклоиды будем менять параметр t , также добавляем малую окружность гипоциклоиды (рис.3.36):

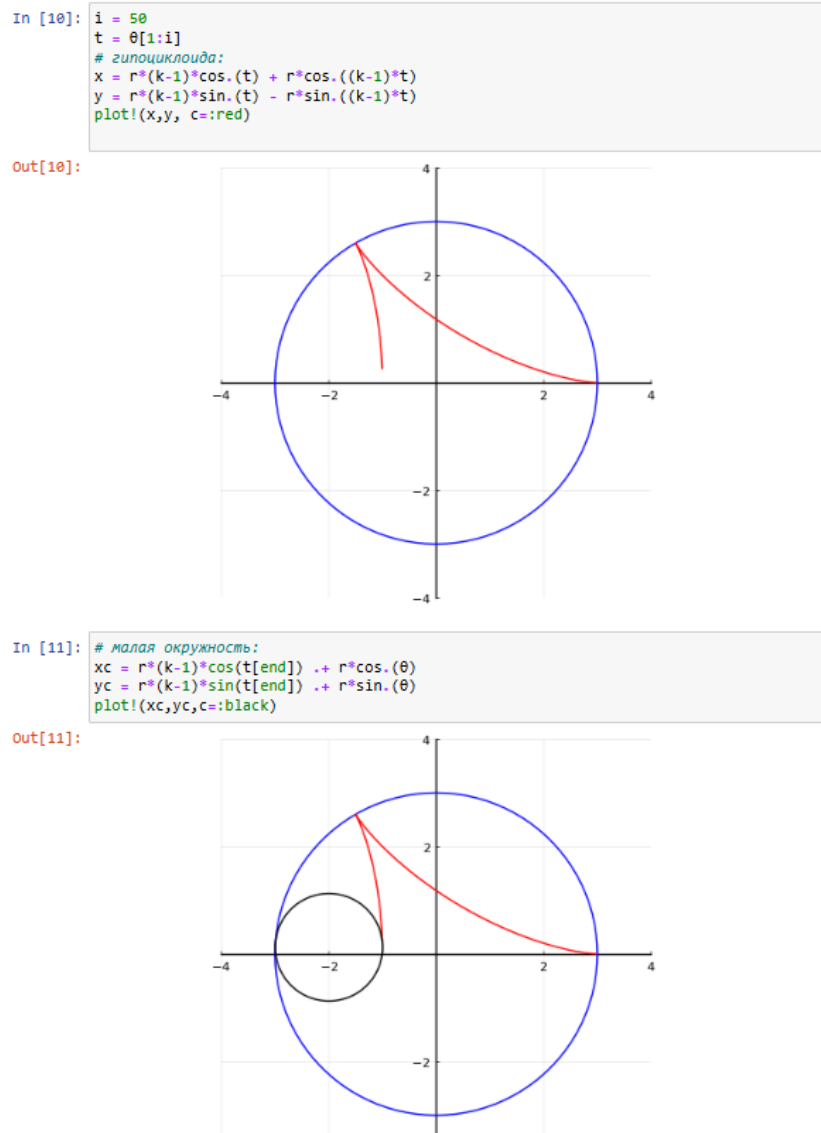


Рис. 3.36: Половина пути гипоциклоиды и малая окружность

Добавим радиус для малой окружности (рис.3.37):

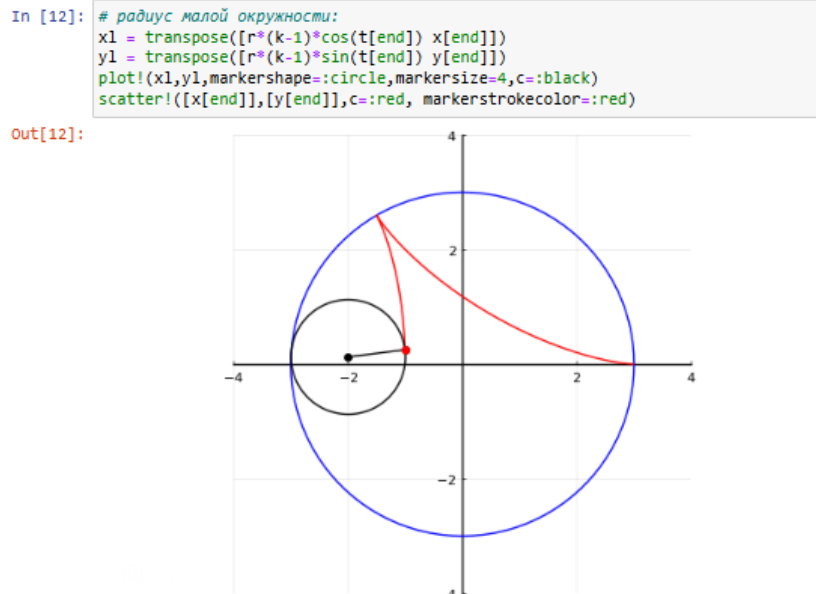


Рис. 3.37: Малая окружность гипоциклоиды с добавлением радиуса

В конце сделаем анимацию получившегося изображения (рис.3.38):

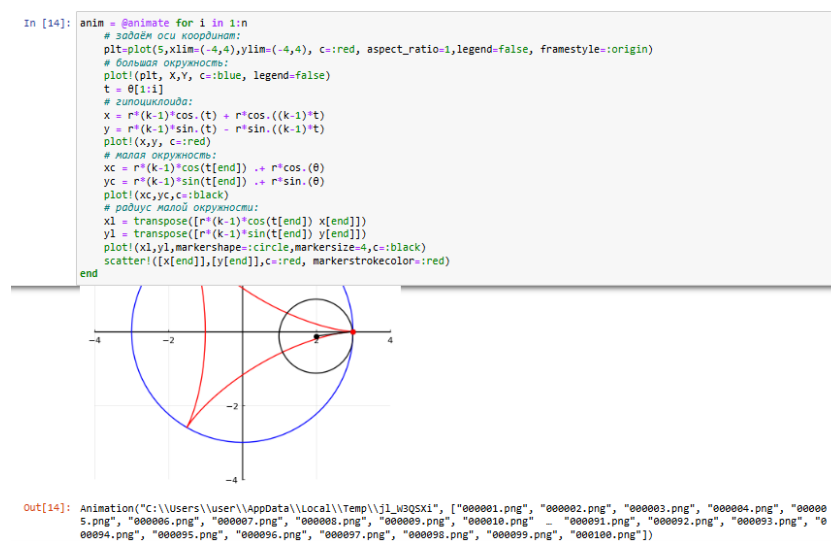


Рис. 3.38: Анимация движения гипоциклоиды

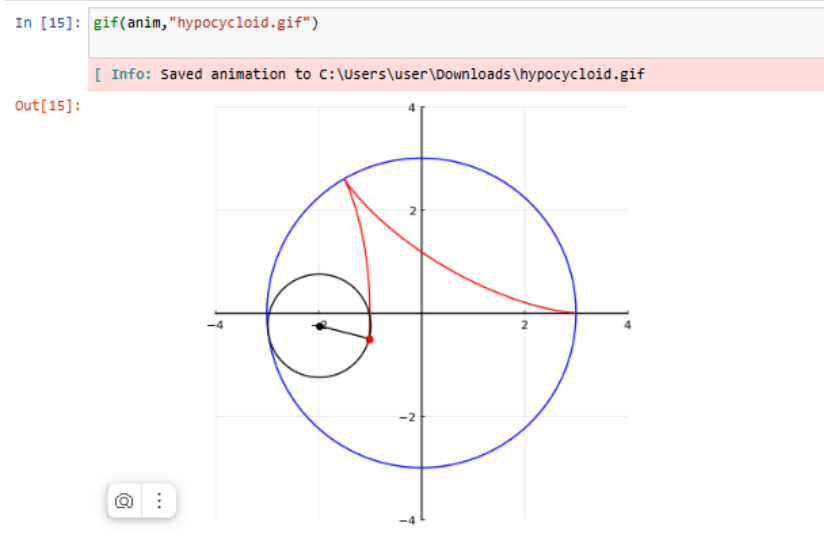


Рис. 3.39: Анимация движения гипоциклоиды

3.1.12 Errorbars

В исследованиях часто требуется изобразить графики погрешностей измерения. Подключим пакет Statistics:

```
## подключение пакета Statistics:
```

```
import Pkg
```

```
Pkg.add("Statistics")
```

```
using Statistics
```

Зададим массив значений, сгенерируем массив ошибок (отклонений от исходных значений) (рис.3.40):

```

In [16]: # подключение пакета Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics

Resolving package versions...
Updating `C:\Users\user\.julia\environments\v1.12\Project.toml`
[10745b16] + Statistics v1.11.1
Manifest No packages added to or removed from `C:\Users\user\.julia\environments\v1.12\Manifest.toml`

In [17]: sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]

Out[17]: 6-element Vector{Float64}:
 1.0
 0.5
 0.25
 0.125
 0.0625
 0.03125

In [18]: n = 10
y = [mean(sd*randn(n)) for sd in sds]
errs = 1.96 * sds / sqrt(n)

Out[18]: 6-element Vector{Float64}:
 0.6198064213930023
 0.3099032106965012
 0.1549516053402506
 0.0774750026741253
 0.03873790133706265
 0.019368950668531323

```

Рис. 3.40: Задание массива значений и генерация массива ошибок

Построим графики исходных значений и исходных значений с отклонениями (рис.3.41):

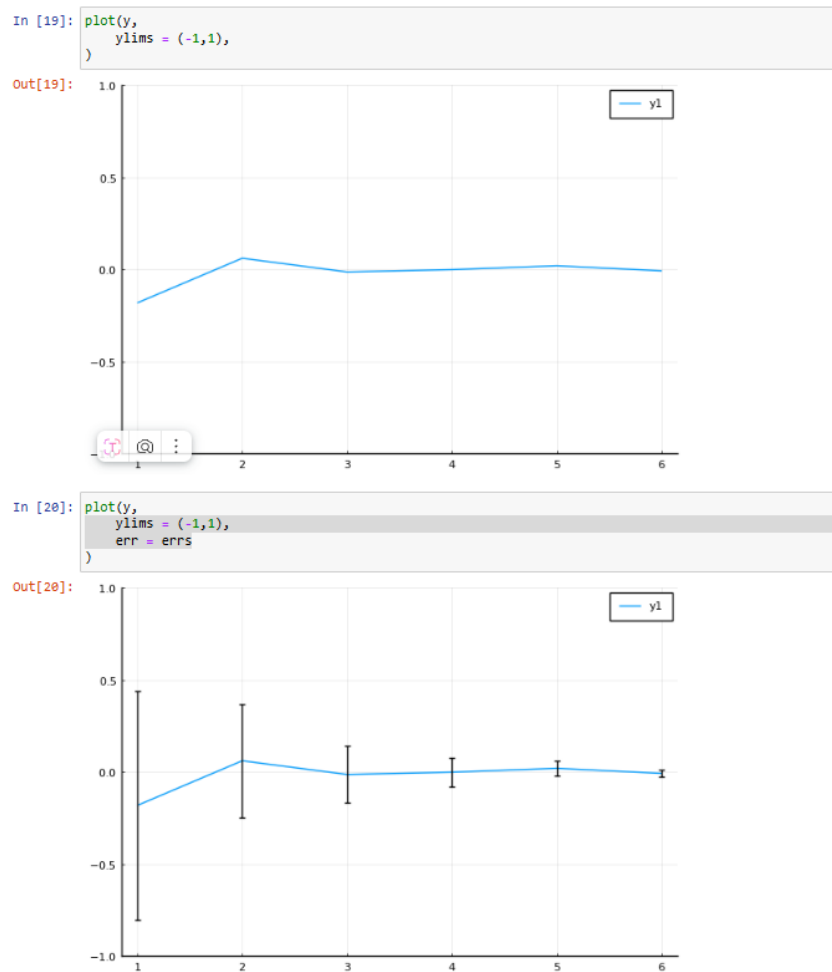


Рис. 3.41: Графики исходных значений и исходных значений с отклонениями

Повернём график (рис.3.42):

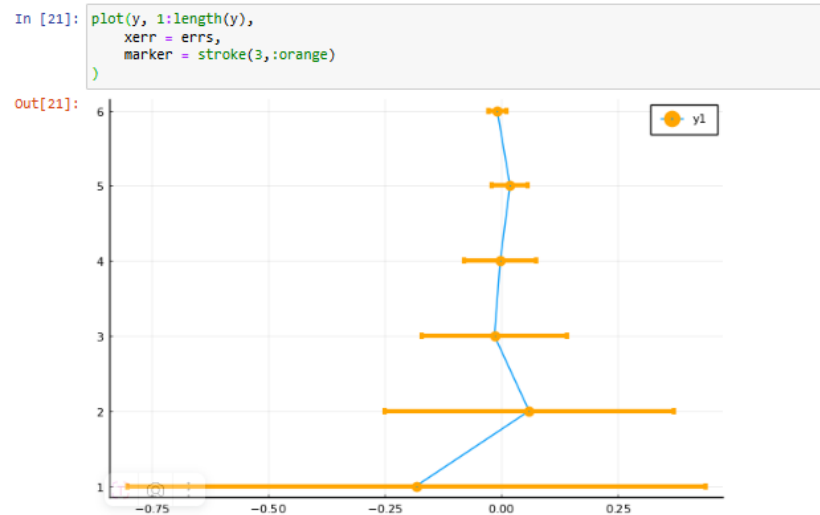


Рис. 3.42: Поворот графика

Заполним область цветом (рис.3.43):

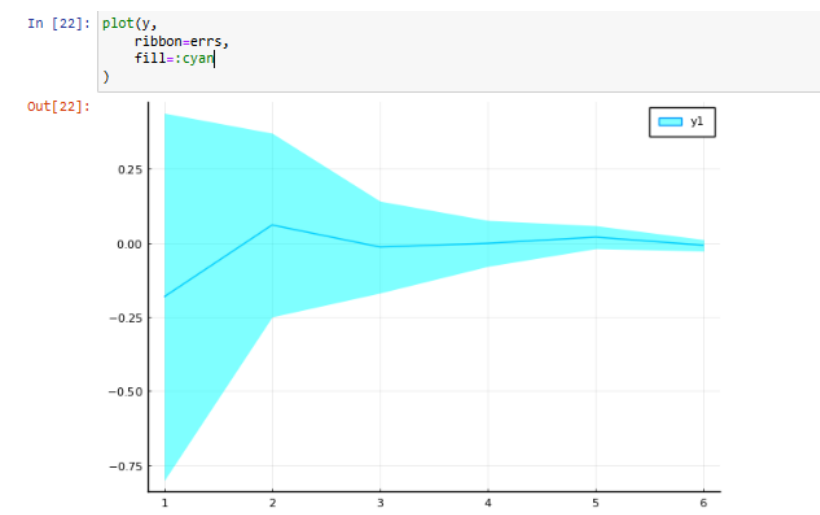


Рис. 3.43: Заполнение цветом

Можно построить график ошибок по двум осям (рис.3.44):

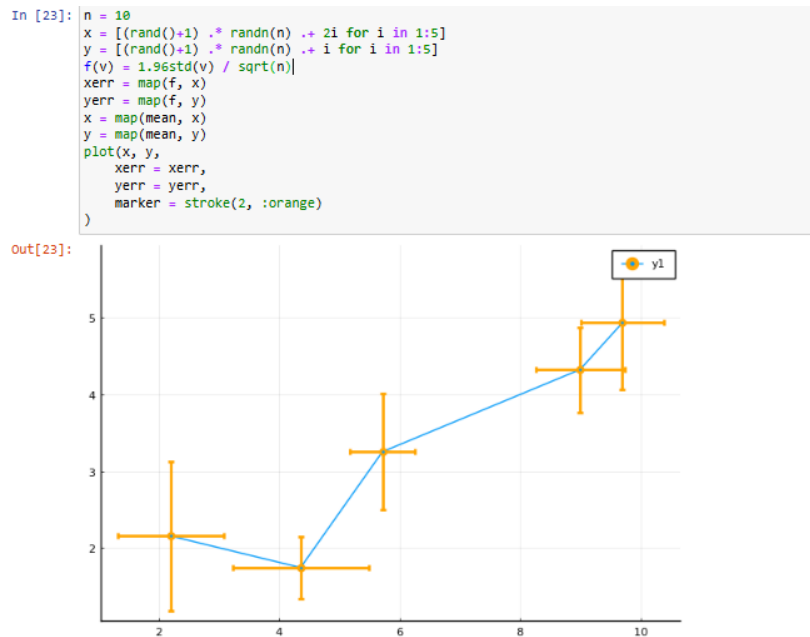


Рис. 3.44: График ошибок по двум осям

Можно построить график асимметричных ошибок по двум осям (рис.3.45):

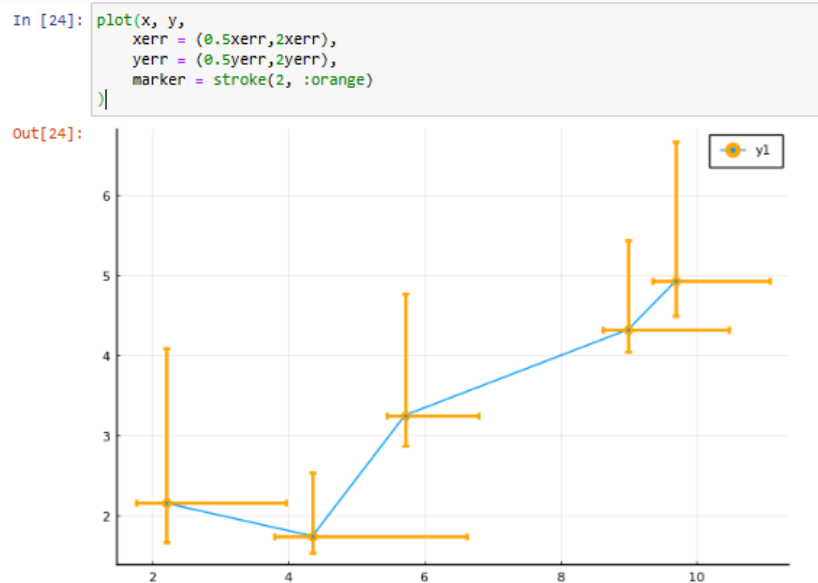


Рис. 3.45: График асимметричных ошибок по двум осям

3.1.13 Использование пакета Distributions

Подгружаем пакет распределений:

```
import Pkg
Pkg.add("Distributions")
using Distributions
```

Построим гистограмму, построенную по массиву случайных чисел (рис.3.46):

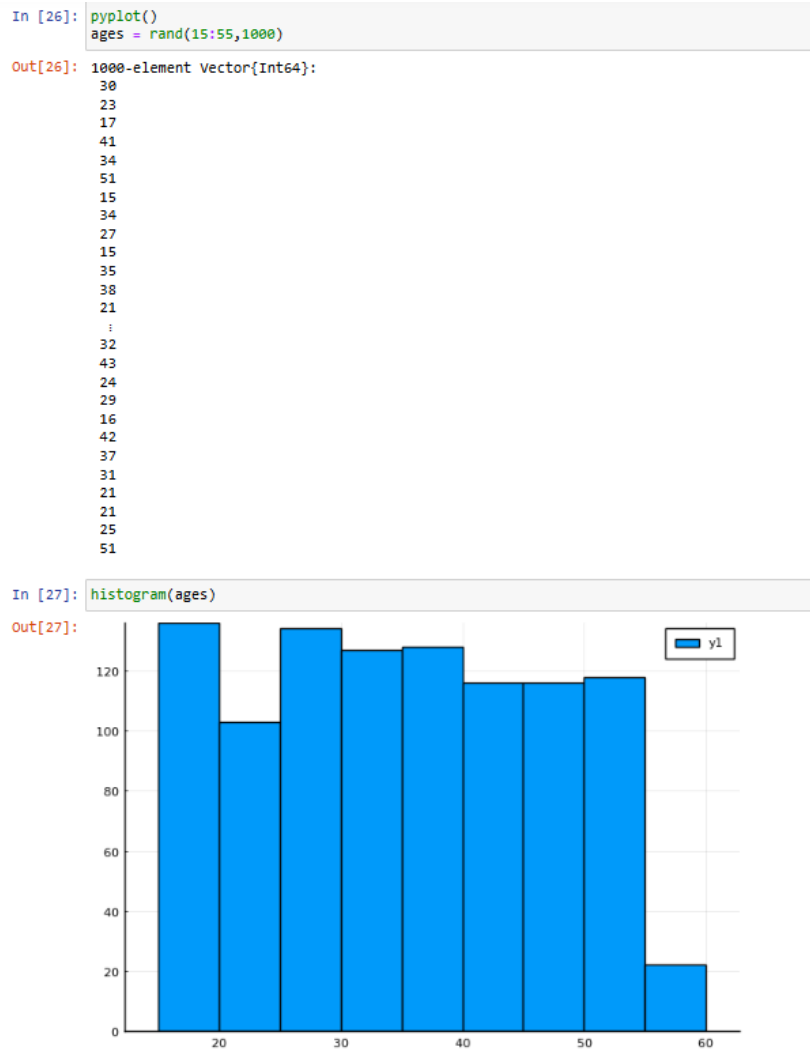


Рис. 3.46: Гистограмма, построенная по массиву случайных чисел

Задаём нормальное распределение и строим гистограмму (рис.3.47):

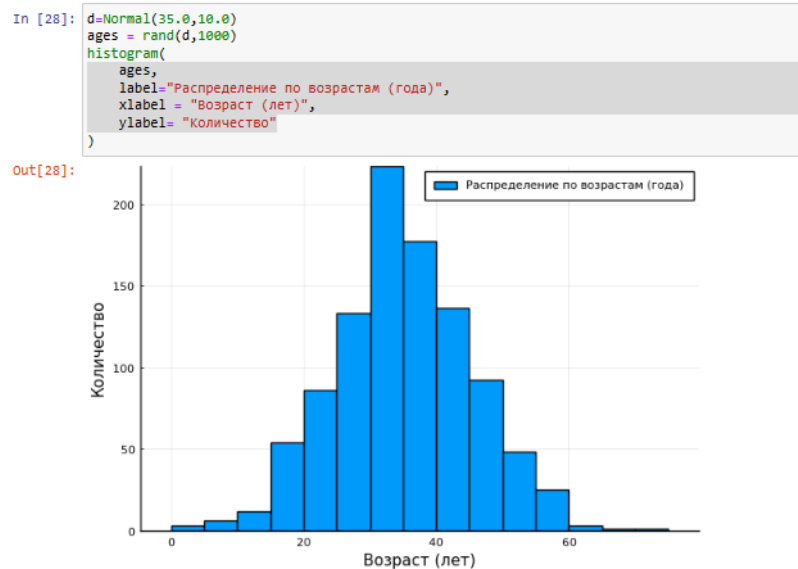


Рис. 3.47: Гистограмма нормального распределения

Далее применим для построения нескольких гистограмм распределения людей по возрастам на одном графике `plotly()` (рис.3.48):

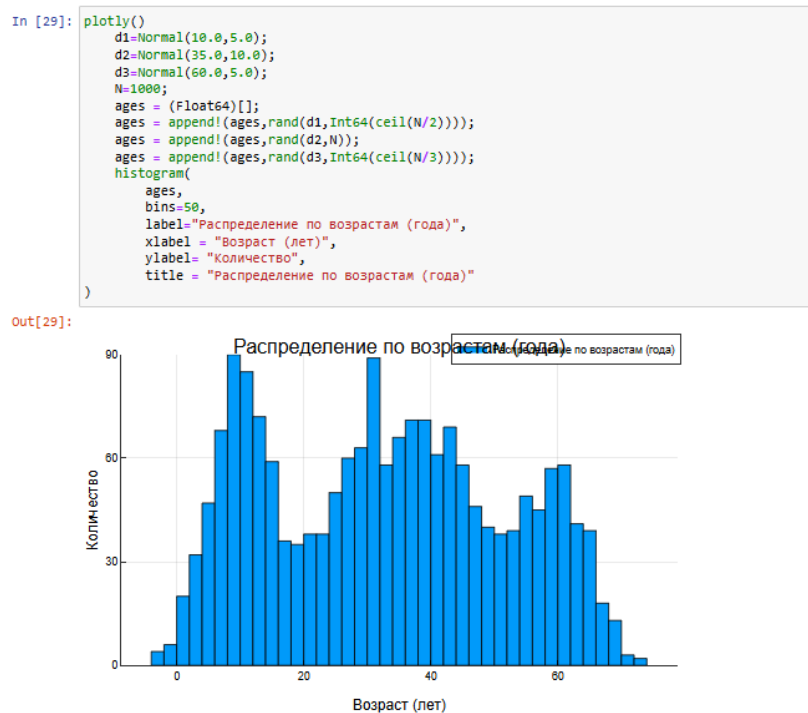


Рис. 3.48: Гистограмма распределения людей по возрастам

3.1.14 Подграфики

Определим макет расположения графиков. Команда `layout` принимает кортеж `layout = (N, M)`, который строит сетку графиков $N \times M$. Например, если задать `layout = (4,1)` на графике четыре серии, то получим четыре ряда графиков (рис.3.49), для автоматического вычисления сетки необходимо передать `layout` целое число (рис.3.50):

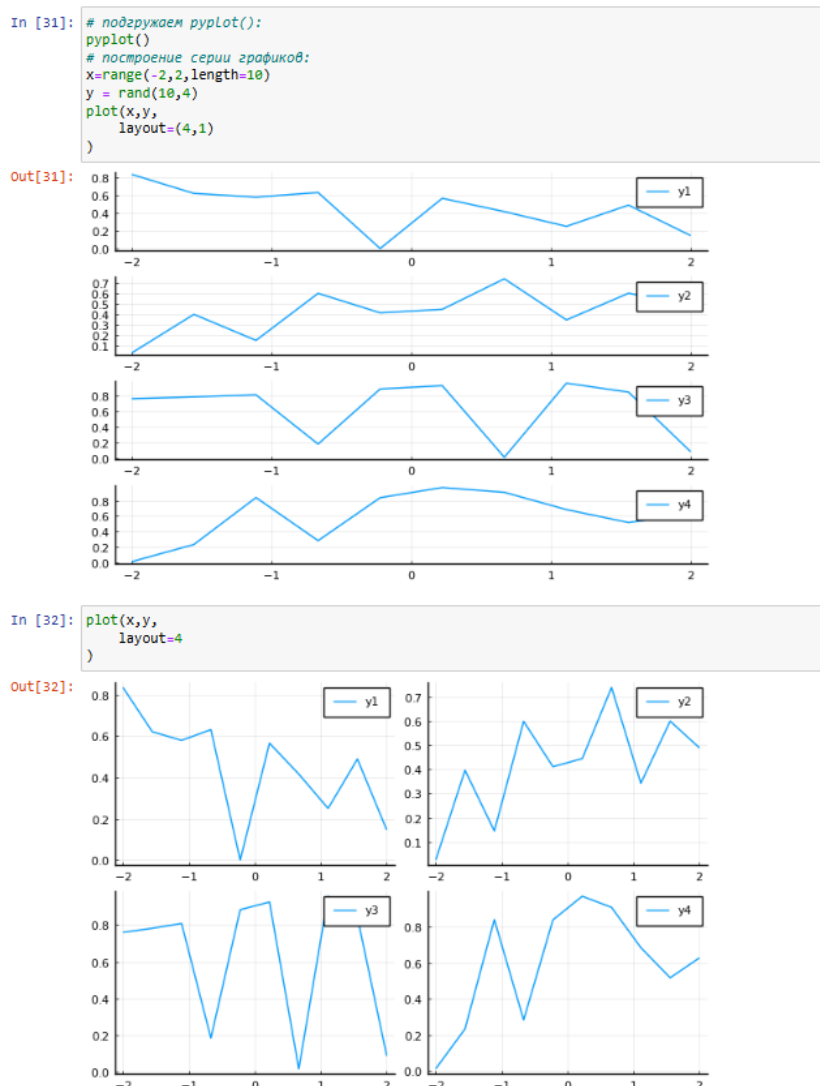


Рис. 3.49: Серия из 4-х графиков в ряд и в сетке

Чуть более сложные макеты сетки могут быть созданы с помощью конструктора `grid(...)`. Например, в следующем макете создаются участки разной высоты

(рис.3.50):

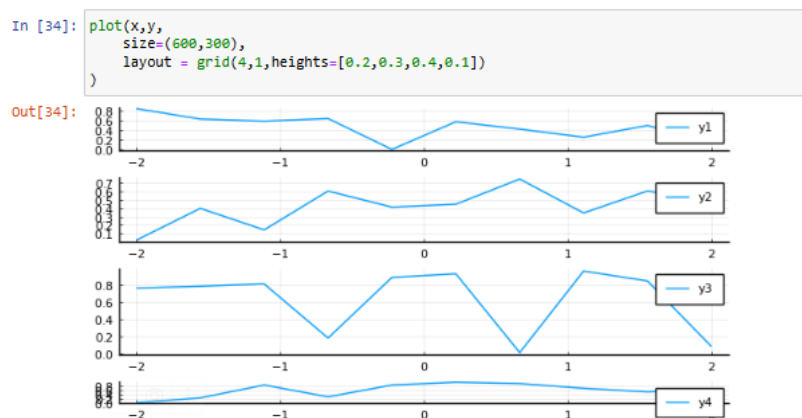


Рис. 3.50: Серия из 4-х графиков разной высоты в ряд

Аргумент `heights` принимает в качестве входных данных массив с долями желаемых высот. Если в сумме дроби не составляют 1,0, то некоторые подзаголовки могут отображаться неправильно.

Можно сгенерировать отдельные графики и объединить их в один, например, в сетке 2×2 (рис.3.51):

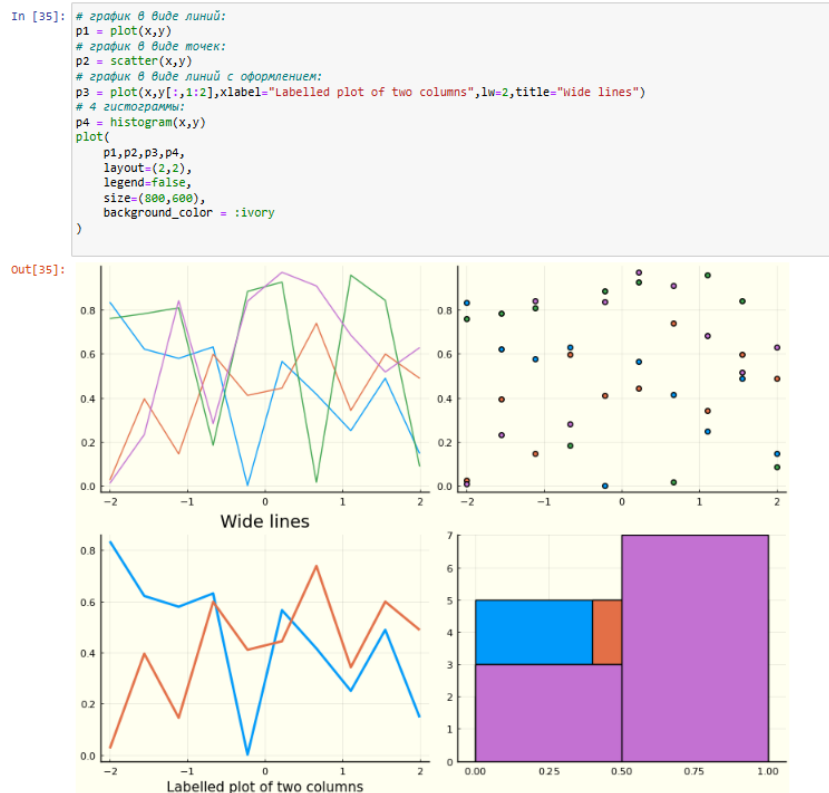


Рис. 3.51: Объединение нескольких графиков в одной сетке

Обратим внимание, что атрибуты на отдельных графиках применяются к отдельным графикам, в то время как атрибуты в последнем вызове `plot` применяются ко всем графикам.

Разнообразные варианты представления данных (рис.3.52):



Рис. 3.52: Разнообразные варианты представления данных

Применение макроса @layout наиболее простой способ определения сложных макетов. Точные размеры могут быть заданы с помощью фигурных скобок, в противном случае пространство будет поровну разделено между графиками (рис.3.53):

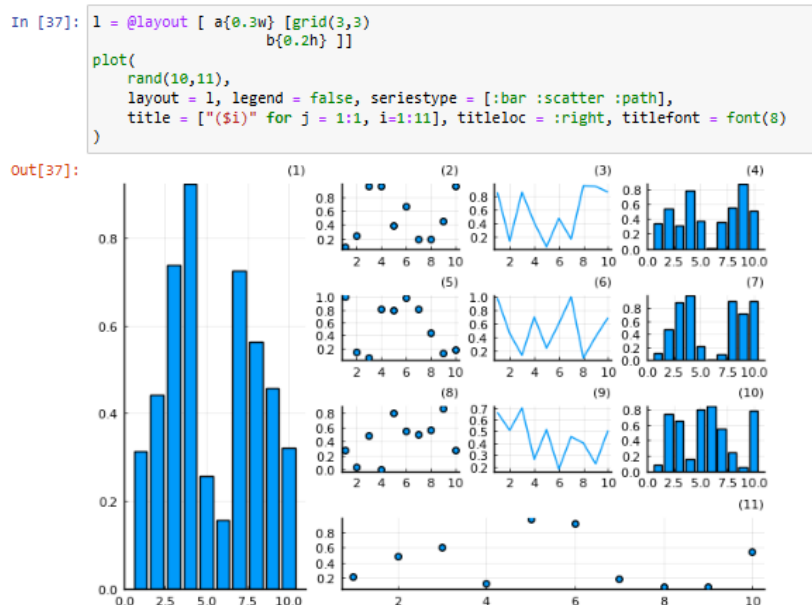


Рис. 3.53: Демонстрация применения сложного макета для построения графиков

3.2 Задания для самостоятельного выполнения

1. Постройте все возможные типы графиков (простые, точечные, гистограммы и т.д.) функции $y = \sin(x)$, где $x \in [0, 2\pi]$. Отобразите все графики в одном графическом окне (рис.3.54):

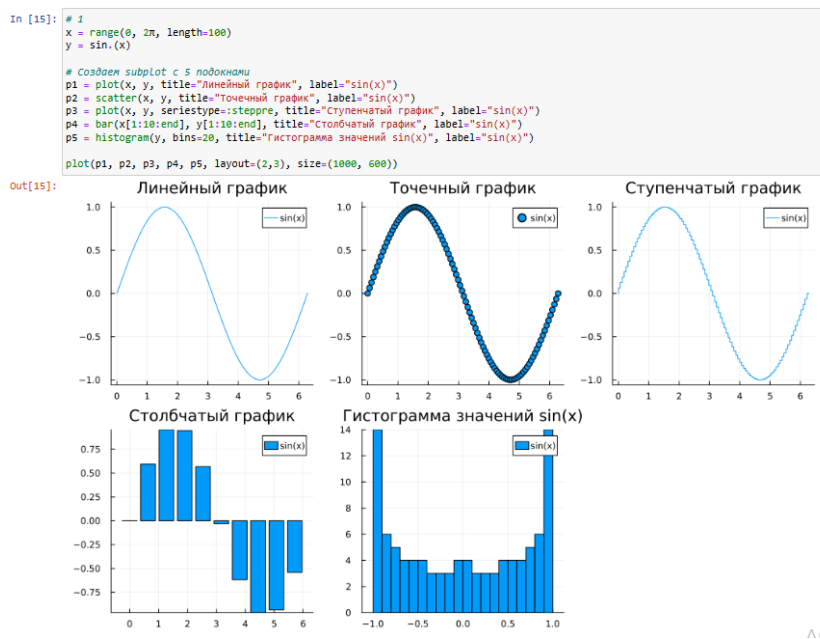


Рис. 3.54: Все возможные типы графиков функции $y = \sin(x)$, где $x \in [0, 2\pi]$

2. Постройте графики функции $y = \sin(x)$, где $x \in [0, 2\pi]$ со всеми возможными (сколько сможете вспомнить) типами оформления линий графика. Отобразите все графики в одном графическом окне (рис.3.55):

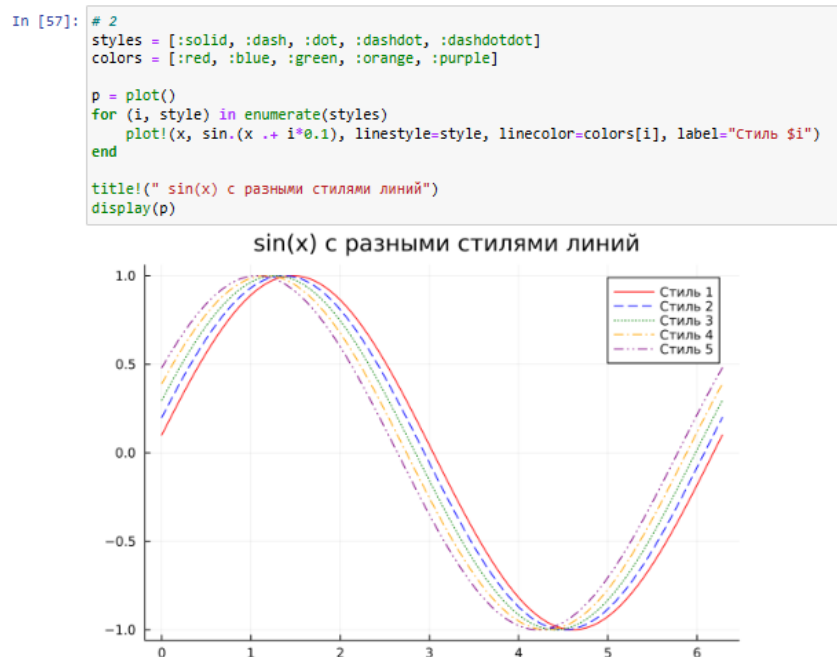


Рис. 3.55: Демонстрация всех типов линий графика для функции $y = \sin(x)$, где $x \in [0, 2\pi]$

3. Постройте график функции $y(x) = \pi x^2 \ln(x)$, назовите оси соответственно. Пусть цвет рамки будет зелёным, а цвет самого графика — красным. Задайте расстояние между надписями и осями так, чтобы надписи полностью умещались в графическом окне. Задайте шрифт надписей. Задайте частоту отметок на осях координат (рис.3.56):

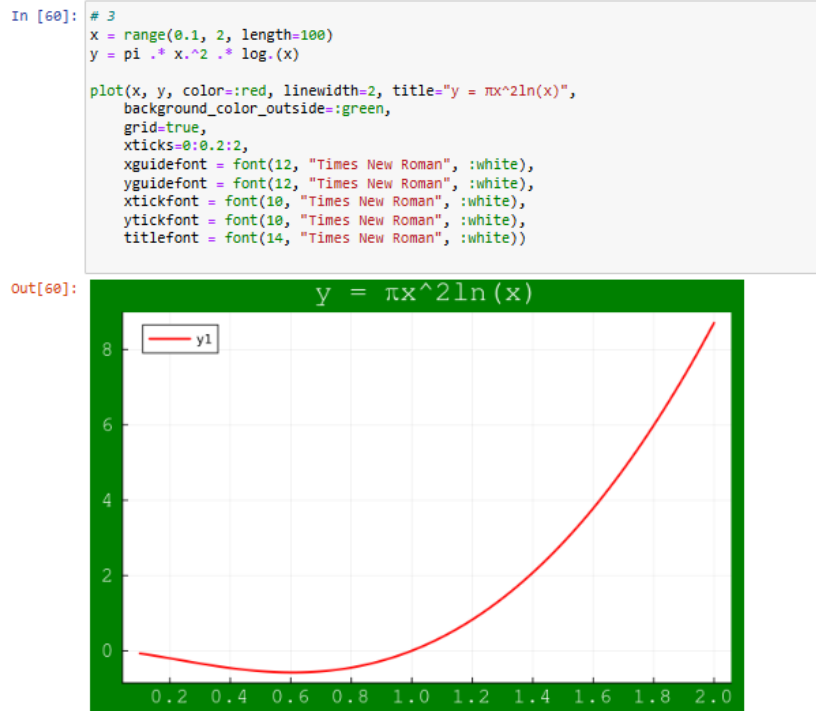


Рис. 3.56: График функции $y(x) = \pi x^2 \ln(x)$

4. Задайте вектор $x = (-2, -2, 0, 1, 2)$. В одном графическом окне (в 4-х подокнах) изобразите графически по точкам x значения функции $y(x) = x^3 - 3x$ в виде:

- точек,
- линий,
- линий и точек,
- кривой.

Сохраните полученные изображения в файле `figure_familiya.png`, где вместо `familiya` укажите вашу фамилию (рис.3.57):

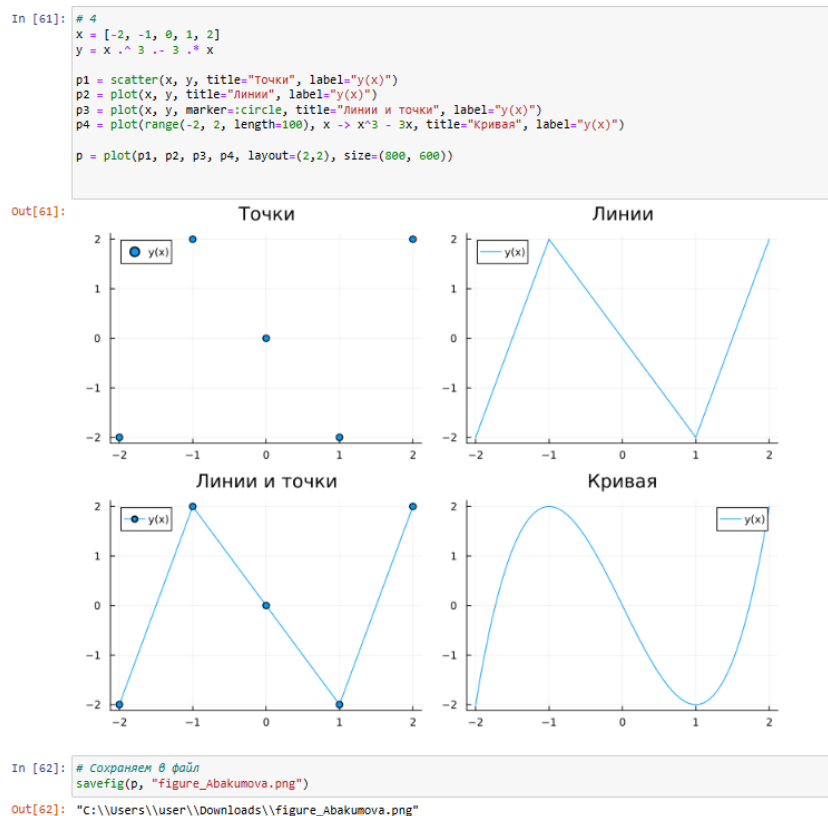


Рис. 3.57: Графическое изображение по точкам x значения функции $y(x) = x^3 - 3x$ в разном виде

5. Задайте вектор $x = (3, 3.1, 3.2, \dots, 6)$. Постройте графики функций $y_1(x) = \pi x$ и $y_2(x) = \exp(x) \cos(x)$ в указанном диапазоне значений аргумента x следующим образом:

- постройте оба графика разного цвета на одном рисунке, добавьте легенду и сетку для каждого графика; укажите недостатки у данного построения (рис.3.58):

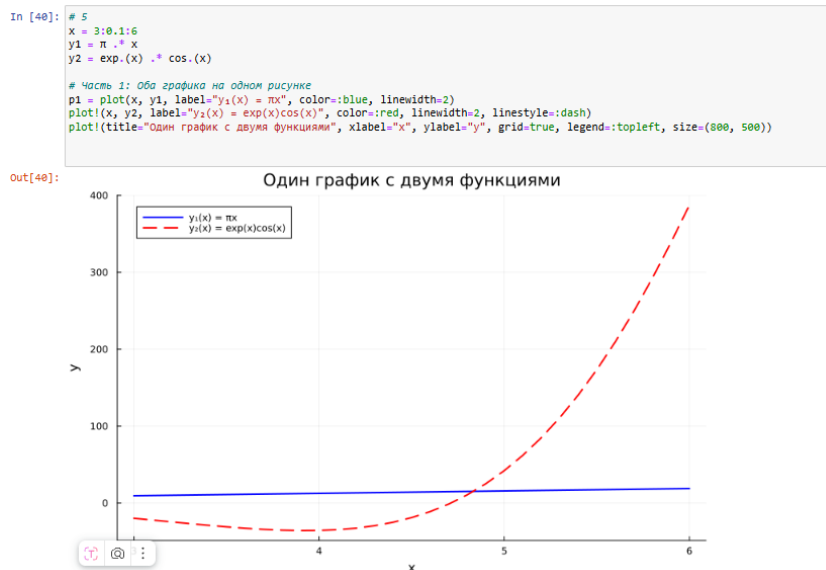


Рис. 3.58: Первое построение графика

Недостатки первого построения:

- Функции имеют разные масштабы ($y_1 \sim 10$, $y_2 \sim \pm 200$), что делает один график почти незаметным.
- Трудно сравнивать поведение функций из-за разных порядков величин.
- постройте аналогичный график с двумя осями ординат (рис.3.59):



Рис. 3.59: Второе построение графика

6. Постройте график некоторых экспериментальных данных (придумайте сами), учитывая ошибку измерения (рис.3.60):

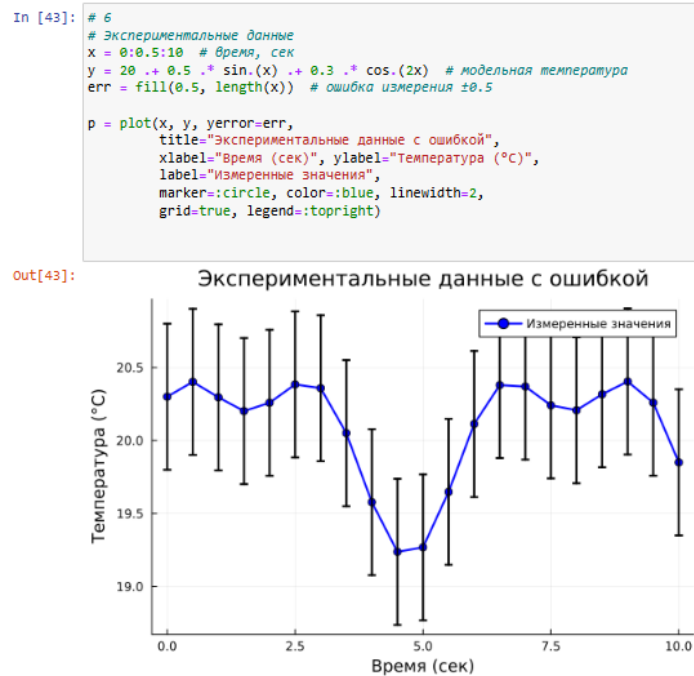


Рис. 3.60: График экспериментальных данных с учетом ошибки измерения

7. Постройте точечный график случайных данных. Подпишите оси, легенду, название графика (рис.3.61):

```
In [63]: # 7
# Генерируем случайные данные
n = 50
x = rand(n)
y = rand(n)

p = scatter(x, y,
            title="Точечный график случайных данных",
            xlabel="X-значения", ylabel="Y-значения",
            label="Случайные точки",
            color=green, markersize=6,
            grid=true, legend=:topleft)
```

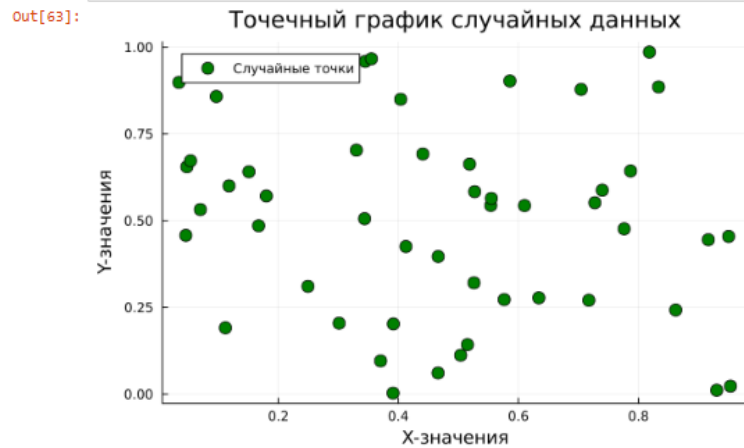


Рис. 3.61: Точечный график случайных данных

8. Постройте 3-мерный точечный график случайных данных. Подпишите оси, легенду, название графика(рис.3.62):

```

In [64]: # 8
# Случайные 3D точки
n = 100
x = rand(n)
y = rand(n)
z = rand(n)

p = scatter(x, y, z,
            title="3D точечный график случайных данных",
            xlabel="X", ylabel="Y", zlabel="Z",
            label="Случайные точки",
            color=:red, markersize=5,
            legend=:bottomleft,
            camera=(30, 30)) # угол обзора

```

Out[64]: 3D точечный график случайных данных

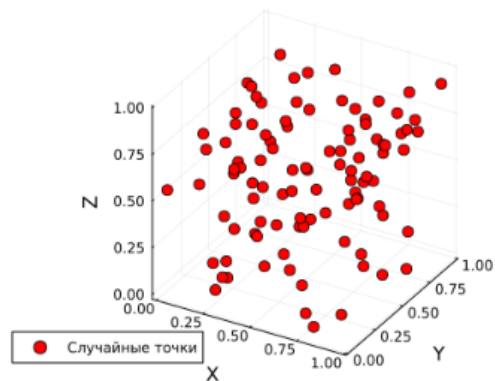


Рис. 3.62: 3-мерный точечный график случайных данных

- Создайте анимацию с построением синусоиды. То есть вы строите последовательность графиков синусоиды, постепенно увеличивая значение аргумента. После соедините их в анимацию (рис.3.63):

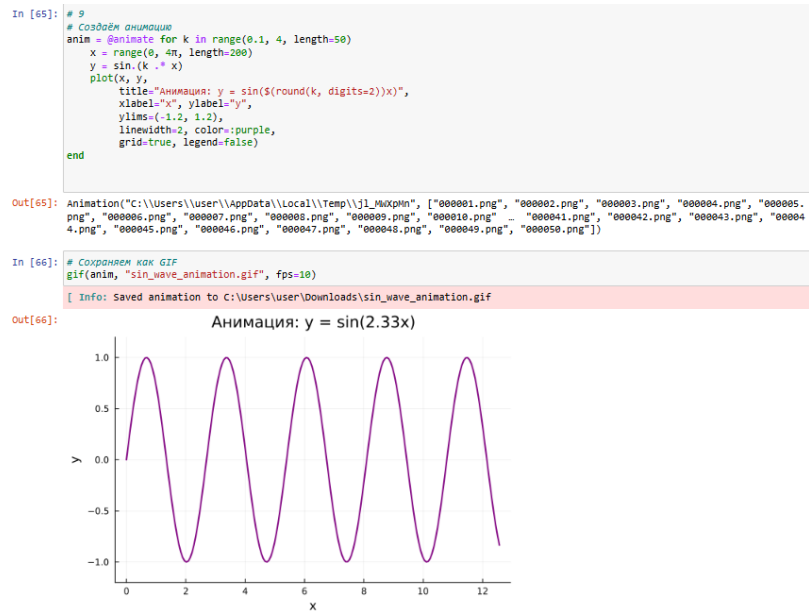


Рис. 3.63: Анимация построения синусоиды

10. Постройте анимированную гипоциклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k (рис.3.64):

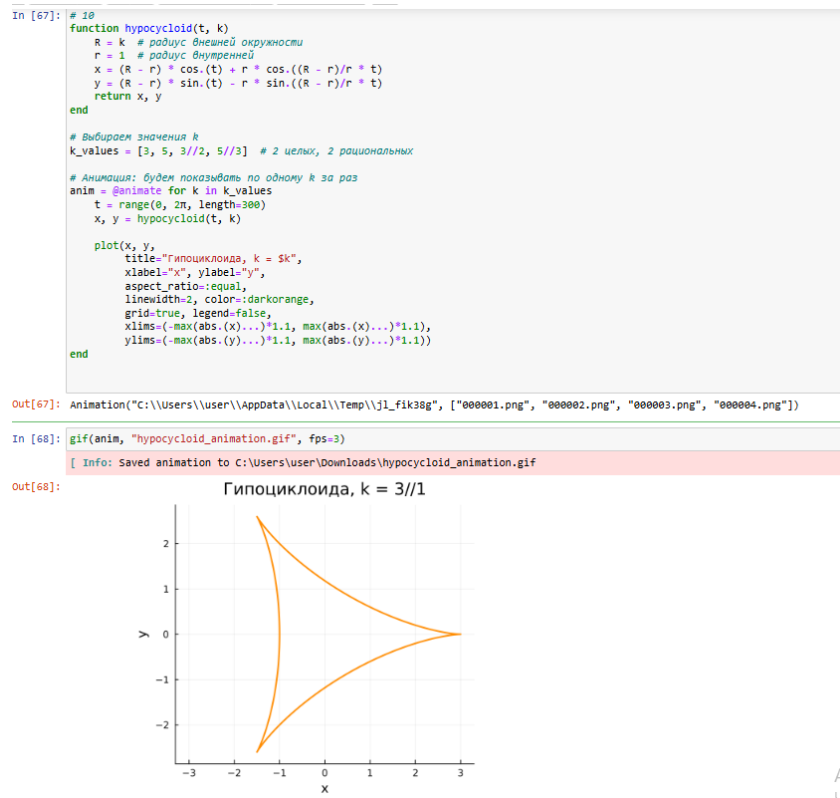


Рис. 3.64: Анимация гипоциклоиды

11. Постройте анимированную эпициклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k (рис.3.65):

```

In [69]: # 11
function epicycloid(t, k)
    R = k
    r = 1
    x = (R + r) * cos.(t) - r * cos.((R + r)/r * t)
    y = (R + r) * sin.(t) - r * sin.((R + r)/r * t)
    return x, y
end

# Те же значения R
k_values = [3, 5, 3//2, 5//3]

anim = @animate for k in k_values
    t = range(0, 2π, length=300)
    x, y = epicycloid(t, k)

    plot(x, y,
         title="Эпициклоида, k = $k",
         xlabel="x", ylabel="y",
         aspect_ratio=:equal,
         linewidth=2, color=:teal,
         grid=true, legend=false,
         xlims=(-max(abs.(x)...)*1.1, max(abs.(x)...)*1.1),
         ylims=(-max(abs.(y)...)*1.1, max(abs.(y)...)*1.1))
end

Out[69]: Animation("C:\Users\user\AppData\Local\Temp\jl_PPP5Jf", ["000001.png", "000002.png", "000003.png", "000004.png"])

In [70]: gif(anim, "epicycloid_animation.gif", fps=3)

[ Info: Saved animation to C:\Users\user\Downloads\epicycloid_animation.gif

```



Рис. 3.65: Анимация эпициклоида

4 Выводы

В результате выполнения данной лабораторной работы я освоила синтаксис языка Julia для построения графиков.